```
      3 still
      3 who
      3 with
      3 xi
alloy@alloy:~/linuxshell/CH17$
```

从上面 topn.sh 脚本的执行结果可以看出，topn.sh 脚本的调用格式如下：

```
./topn.sh n filename                    #第 1 个参数 n 是查找最高频率单词的数量
                                        #第 2 个参数 filename 是需要统计单词频率的目标文件
```

poem.txt 中出现频率最高的单词是 the，出现了 14 次，对于出现频率相同的单词，topn.sh 根据字母顺序对单词进行排序，如例 17-2 中，出现频率为 3 的单词，以字母 a 开头的单词排在最前面，以字母 x 开头的单词排在最后面。topn.sh 脚本十分简捷，但是，它综合应用了管道、tr、sort、uniq、head 等文本处理命令。同时，本节的例子也充分说明了 Linux Shell 工具在文本处理方面是非常强大的。

17.3 伪随机数的产生和应用

计算机不会产生绝对随机的随机数，它只能产生“伪随机数”。其实绝对随机的随机数只是一种理想的随机数，不管计算机怎样发展，它也不会产生一串绝对随机的随机数，只能生成相对的随机数，即伪随机数。

伪随机数并不是假随机数，这里的“伪”是有规律的意思，就是计算机产生的伪随机数既是随机的，又是有规律的。怎样理解呢？产生的伪随机数有时遵守一定的规律，有时不遵守任何规律；伪随机数有一部分遵守一定的规律；另一部分不遵守任何规律。比如，“世上没有两片形状完全相同的树叶”，这正是说明了事物的特性，即随机性，但是每种树的叶子都有近似的形状，这正是事物的共性，即规律性。同理，我们就容易理解：计算机只能产生伪随机数，不能产生绝对随机的随机数。严格地说，这里的计算机是指由冯·诺依曼思想发展起来的电子计算机，而未来的量子计算机有可能产生基于自然规律的不可重现的“真”随机数。

C/C++、Java 等高级程序设计语言都有产生伪随机数的工具，bash Shell 同样也定义了伪随机数产生工具，即$RANDOM 函数，每次调用这个函数将返回一个伪随机整数，范围在 0～32767 之间。下面先通过一个最简单的例子说明$RANDOM 函数的基本用法，例 17-3 编写了 random.sh 脚本用于产生 5 个随机数：

```
#例 17-3：random.sh 脚本产生 5 个随机数
#!/bin/bash

MAX=5                                   #产生随机数的总量
i=1                                     #计数器，初值是 1

echo "$MAX random numbers are generated:"
while [ "$i" -le $MAX ]
do
 number=$RANDOM                         #调用$RANDOM 产生随机数
 echo $number
```

```
 let "i=i+1"                                          #计数器增加 1
done
```

random.sh 脚本利用一个 while 循环产生 5 个随机数，while 循环体内调用$RANDOM 函数，每次调用产生不同的随机数，下面给出 random.sh 脚本的执行结果：

```
#例 17-3 random.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH17$ chmod u+x random.sh
alloy@alloy:~/linuxshell/CH17$ ./random.sh  #执行 random.sh 两次得到不同的结果
5 random numbers are generated:
12208
22841
15794
29988
8770
alloy@alloy:~/linuxshell/CH17$ ./random.sh
5 random numbers are generated:
12760
31177
4923
15889
30836
alloy@alloy:~/linuxshell/CH17$
```

我们执行 random.sh 脚本两次得到不同的随机数序列。如果我们需要产生在特定范围内的随机数，一般使用取模操作，返回除法的余数，余数必定是小于被除数的。

随机数在互联网中具有广泛的应用背景，如计算机仿真模拟、数据加密、网络游戏等。本节选取两个随机数的应用例子来说明$RANDOM 函数在 Shell 脚本编程中的应用。在登录某些论坛或游戏时，系统经常会产生一个由随机数字和字母组成的图片，用户在进行下一步操作之前必须正确地输入图片中的数字和字母，这是一个防止恶意攻击的很好的方法，因为黑客难以破解出图片格式的字符，图 17-2 显示了登录 QQ 游戏时遇到的图片。

图 17-2　QQ 游戏的图片验证方式

实现图片验证的关键技术就是产生随机数，下面的脚本 seqrand.sh 用于生成一段定长的随机字符串，字符串由数字和大小写字母组成，seqrand.sh 脚本的内容如下：

```
#例 17-4：seqrand.sh 脚本产生定长的随机字符串
#!/bin/bash

length=6                                          #随机字符串的长度
i=1                                               #计数器，初值是 1

#seq 是数字、大小写字母的序列
seq=(0 1 2 3 4 5 6 7 8 9 a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D
```

```
E F G H I J K L M N O P Q R S T U V W X Y Z)

    num_seq=${#seq[@]}                                  #计算 seq 数组的长度

    while [ "$i" -le "$length" ]                        #产生$length 个随机字符
    do
     seqrand[$i]=${seq[$((RANDOM%num_seq))]}            #取模产生 num_seq 内的随机数
     let "i=i+1"
    done

    #下面是输出结果
    echo "The random string is:"
    for j in ${seqrand[@]}
    do
     echo -n $j
    done
    echo
```

seqrand.sh 脚本首先定义了几个变量：length 表示随机字符串的长度，i 表示计数器，初值为 1，seq 是一个数组变量，保存了数字和大小写字母的序列，seq 是随机字符串产生的全集。然后，seqrand.sh 脚本计算 seq 数组的长度，赋给 num_seq。while 循环是脚本的主体，用于产生$length 个随机字符，其中的核心语句 seqrand[$i]=${seq[$((RANDOM%num_seq))]} 值得寻味，它使用双圆括号结构计算随机数 RANDOM 与 num_seq 的取模运算，将取模的结果作为索引去除 seq 数组中对应的值，赋给 seqrand 数组。当 while 循环结束时，seqrand 数组即可保存随机字符串。最后，seqrand.sh 脚本依次输出 seqrand 数组，得到所产生的随机字符串。下面给出例 17-4 中 seqrand.sh 脚本的执行结果：

```
#例 17-4 seqrand.sh 脚本的执行结果
#连续三次执行 seqrand.sh 脚本，得到不同的随机字符串
alloy@alloy:~/linuxshell/CH17$ chmod u+x seqrand.sh
alloy@alloy:~/linuxshell/CH17$ ./seqrand.sh
The random string is:
Mrl8an
alloy@alloy:~/linuxshell/CH17$ ./seqrand.sh
The random string is:
4AGHNC
alloy@alloy:~/linuxshell/CH17$ ./seqrand.sh
The random string is:
uaWA3J
alloy@alloy:~/linuxshell/CH17$
```

我们连续三次执行 seqrand.sh 脚本，得到不同的随机字符串。我们可以利用 ASP.NET 等工具将这些随机字符串封装成图片格式以作为验证图片了。

网络游戏中也经常需要利用随机数完成一些功能，比如掷骰子、发扑克牌等。下面我们举一个掷骰子的例子，dice.sh 脚本连续掷 1000 次骰子，然后统计出 1～6 点的次数，dice.sh 脚本的内容如下：

```
#例 17-5：dice.sh 脚本模拟投骰子游戏
#!/bin/bash

PIPS=6                                  #一个骰子有 6 面
MAX=1000                                #投骰子的次数
throw=1                                 #计数器，初值是 1
```

```
#下面是 6 个计数的变量
one=0
two=0
three=0
four=0
five=0
six=0

count()                                  #更新计数的次数
{
case "$1" in
 0) let "one=one+1";;                    #跟$PIPS 变量取模为 0～5，因此，0 代表骰子的 1
 1) let "two=two+1";;
 2) let "three=three+1";;
 3) let "four=four+1";;
 4) let "five=five+1";;
 5) let "six=six+1";;
esac
}

while [ "$throw" -le "$MAX" ]            #开始掷骰子
do
  let "dice=RANDOM % $PIPS"              #dice 是 0～5 之间的随机数
  count $dice                            #更新统计次数的值
  let "throw=throw+1"
done

#下面输出统计结果
echo "The statistics results are as follows:"
echo "one=$one"
echo "two=$two"
echo "three=$three"
echo "four=$four"
echo "five=$five"
```

dice.sh 脚本定义 count 函数，根据位置参数对掷骰子的点数进行统计，骰子有 6 个面，PIPS=6，random % $PIPS 得出的值就在 0～5 之间。因此，用 count 函数进行简单转换，0 对应于骰子的 1，以此类推。dice.sh 脚本的 while 循环掷$MAX 次骰子，每得到一个骰子值，就更新统计次数，最后，输出 6 个统计变量的值。下面给出例 17-5 中 dice.sh 脚本的执行结果：

```
#例 17-5 dice.sh 脚本的执行结果
#dice 脚本的执行结果
alloy@alloy:~/linuxshell/CH17$ chmod u+x dice.sh
alloy@alloy:~/linuxshell/CH17$ ./dice.sh
The statistics results are as follows:
one=189                                  #投到 1 点的次数是 189
two=161
three=163
four=147
five=184
six=156
alloy@alloy:~/linuxshell/CH17$
```

从上面的 dice.sh 脚本执行结果可以看出，random 函数产生的随机数分布还是比较平均

的，1000 次掷骰子，平均每个面应被掷到 166.7 次，结果基本在平均值左右浮动（即方差较小）。总之，随机数是编程的常用手段，其应用极为广泛，bash Shell 编程提供了产生伪随机数的函数——random 函数，它能方便地产生分布较平均的伪随机数，能满足大部分应用的需求。

17.4 crontab 的设置和应用

crontab 命令的功能是以一定的时间间隔调度一些命令的执行。在/etc 目录下有一个 crontab 文件，这里存放有系统运行的一些调度程序。每个用户可以建立自己的调度 crontab，我们可以使用下面两条命令来编辑 crontab 调度表：

```
crontab -u user -e
vim /etc/crontab                    #与上面两条命令是等价的
```

由于每个用户都有自己的 crontab，因此，编辑 crontab 调度表需要用-u user 来指定用户；而 vim 命令则是编辑当前用户的 crontab 调度表。crontab 命令-e 选项表示编辑 crontab 调度表，除了-e 选项之外，crontab 命令还有其他几个选项，我们将 crontab 命令的选项及其意义列于表 17-1 中。

表 17-1　crontab 命令的选项及其意义

选项名称	意　义
e	编辑 crontab 调度表
l	在 stdout 上显示 crontab
r	删除当前的 crontab 文件

Linux 系统还定义了两个文件来控制 crontab，它们是：/etc/cron.allow 和/etc/cron.deny，/etc/cron.allow 表示哪些用户能使用 crontab 命令，如果它是一个空文件，表明没有一个用户能安排作业。如果这个文件不存在，而有另外一个文件/etc/cron.deny，则只有不包括在这个文件中的用户才可以使用 crontab 命令。如果它是一个空文件，表明任何用户都可安排作业。两个文件同时存在时，cron.allow 优先，如果都不存在，只有超级用户可以安排作业。Linux 系统用户的作业与它们预定的时间存储在文件/usr/spool/cron/crontabs/username 里。username 使用户名，在相应的文件中存放着该用户所要运行的命令。命令执行的结果无论是标准输出还是错误输出，都将以邮件形式发给用户。

下面的例 17-6 中的命令显示了 root 用户的 crontab，由于目前还没有为 root 用户创建 crontab，/etc/crontab 文件中只有默认的代码和注释。

```
#例 17-6：显示了 alloy 用户的 crontab
alloy@alloy:/etc$ crontab -u alloy -l
no crontab for alloy
alloy@alloy:/etc$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
```

```
    # that none of the other crontabs do.

    SHELL=/bin/sh
    PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

    # m h dom mon dow user  command
    17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
    25 6    * * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report
/etc/cron.daily )
    47 6    * * 7   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report
/etc/cron.weekly )
    52 6    1 * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report
/etc/cron.monthly )
    #
    alloy@alloy:~/linuxshell/CH17$
```

从例 17-6 中/etc/crontab 文件的内容可以看出，crontab 中的语句格式为：

```
* * * * * 用户名 可执行命令
```

前面的五个“*”分别表示“分钟、小时、日期、月份、星期”，取值范围分别为“0～59、0～23、1～31、1～12、0～6”，比如：

```
* * * * *                          #表示每分钟
1 * * * *                          #表示每小时第 1 分钟
02 12 * * *                        #表示每天 12 点第 2 分钟（每天 12：02）
0-59/2 * * * *                     #表示每 2 分钟执行一次任务
```

crontab 在 Linux 系统管理方面有着广泛的应用，同时，crontab 也是用户定制 Shell 环境的常用工具，下面举一个实例来说明 crontab 的应用。

例 17-7 的问题是：某系统管理员需每天做一定的重复工作，请按照下列要求，编制一个解决方案，步骤如下：

① 在下午 4:50 删除/abc 目录下的全部子目录和全部文件。

② 从早 8:00 到下午 6:00，每小时读取/xyz 目录下 x1 文件中每行第一个域的全部数据加入到/backup 目录下的 bak01.txt 文件内。

③ 每逢星期一下午 5:50，将/data 目录下的所有目录和文件归档并压缩为文件 backup.tar.gz。

④ 在下午 5:55 将 IDE 接口的 CD-ROM 卸载（假设 CD-ROM 的设备名为 hdc）。

⑤ 在早晨 8:00 前开机后启动。

例 17-7 中要求重复的系统管理工作可以利用 crontab 很方便地完成，我们可以在/etc/crontab 文件中添加下列语句来完成：

```
#例 17-7：重复工作的解决方案
alloy@alloy:~/linuxshell/CH17$ cat /etc/crontab
SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
HOME=/
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7)  OR
```

```
#sun,mon,tue,wed,thu,fri,sat
# |  |  |  |  |
# *  *  *  *  *  command to be executed

50 16 * * * rm -r /abc/*                                    #完成目标①
0 8-18/1 * * * cut -f1 /xyz/x1 >;>; /backup/bak01.txt      #完成目标②
50 17 * * * tar zcvf backup.tar.gz /data                    #完成目标③
55 17 * * * umount /dev/hdc                                 #完成目标④

alloy@alloy:~/linuxshell/CH17$
```

我们在 crontab 中加入四条语句，分别对应于例 17-7 中的①～④个目标，但是，目标⑤无法在 crontab 中完成，这需要在每日早晨 8:00 之前开机后启动 crontab 即可。

最后，再举一个利用 crontab 实现定时文件备份的例子，比如，系统管理员在每月第一天备份并压缩/etc 目录的所有内容，存放在/root/bak 目录里，且文件名为 yymmdd_etc 形式，其中，yy 为年，mm 为月，dd 为日。我们可以编写一个脚本实现备份功能，然后在 crontab 中设定每天执行该脚本来达到上述目标。备份脚本的名字为 fileback.sh，内容如下：

```
#例 17-8：fileback.sh 脚本用于将/etc 目录的所有内容进行备份
#!/bin/bash

DIRNAME=`ls /root | grep bak`                               #获取/root/bak 字符串

if [ -z "$DIRNAME" ]                                        #如果/root/bak 不存在，则创建一个
then
mkdir /root/bak
cd /root/bak
fi

#获取当前年、月、日数据存储到 YY、MM、DD 变量中
YY=`date +%y`
MM=`date +%m`
DD=`date +%d`

BACKETC=$YY$MM$DD_etc.tar.gz                                #备份文件的名字
tar zcvf $BACKETC /etc                                      #将/etc 所有的文件打包
echo "fileback finished!"
```

fileback.sh 脚本首先试图获取/root/bak 字符串，如果该目录不存在，则立刻创建该目录。然后，fileback.sh 脚本利用 date 命令获取当前年、月、日数据存储到 YY、MM、DD 变量中，利用这三个变量组成备份文件的名字，并存储到 BACKETC 变量中，最后，fileback.sh 脚本利用 tar 命令将/etc 所有的文件打包放到 BACKETC 文件中。

我们只要在/etc/crontab 中添加语句，并且每天执行 fileback.sh 脚本一次，就可以完成定期备份功能。

```
#例 17-8 fileback.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH17$ cat /etc/crontab
SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
HOME=/
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
```

```
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7)  OR
#sun,mon,tue,wed,thu,fri,sat
# |  |  |  |  |
# *  *  *  *  *  command to be executed

0 1 * * * /bin/bash /usr/bin/fileback.sh              #每天执行 fileback.sh 脚本一次
alloy@alloy:~/linuxshell/CH17$
```

例 17-8 的思路可以扩展到很多实际的例子中，先编写一个实现基本操作的脚本，再利用 crontab 定时执行它。

17.5 使用 MySQL 数据库

17.5.1 MySQL 基础

数据库能永久存储各种数据，Shell 脚本经常需要从数据库中读取数据、处理数据，再将结果写回到数据库，Linux 系统中常用的数据库有 MySQL、PostgreSQL、Oracle、DB2 等，Shell 脚本使用这些数据库的方法大同小异，本节以 MySQL 数据库管理系统为例介绍 Shell 编程与 MySQL 的结合使用。

MySQL 是一个高性能的关系型数据库管理系统，具有功能强大、灵活性好的应用编程接口（API）和精巧的系统结构。MySQL 是现今世界上最受欢迎的开放源代码数据库，受到了广大软件用户的青睐。由于体积小、速度快、总体拥有成本低，尤其是开放源代码这一特点，Internet 上的许多中小型网站都选择 MySQL 作为网站数据库。

在 Ubuntu 12.04 下安装 MySQL 的方法非常简单，直接使用 apt-get 工具即可，输入 sudo apt-get install mysql-server，此时会出现对应的提示。

```
alloy@alloy:~/linuxshell/CH17$ sudo apt-get install mysql-server
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
将会安装下列额外的软件包：
  libdbd-mysql-perl libdbi-perl libhtml-template-perl libmysqlclient18 libnet-daemon-
perl libplrpc-perl libterm-readkey-perl
  mysql-client-5.5 mysql-client-core-5.5 mysql-common mysql-server-5.5 mysql-server-
core-5.5
建议安装的软件包：
  libipc-sharedcache-perl tinyca mailx
下列【新】软件包将被安装：
  libdbd-mysql-perl libdbi-perl libhtml-template-perl libmysqlclient18 libnet-daemon-
perl libplrpc-perl libterm-readkey-perl
  mysql-client-5.5 mysql-client-core-5.5 mysql-common mysql-server mysql-server-5.5
mysql-server-core-5.5
升级了 0 个软件包，新安装了 13 个软件包，要卸载 0 个软件包，有 4 个软件包未被升级。
需要下载 27.3 MB 的软件包。
解压缩后会消耗 97.5 MB 的额外空间。
```

```
    您希望继续执行吗？[Y/n]
```

此时使用“Y”来确认继续，Ubuntu 即开始自动下载和安装 MySQL，当安装完成之后会自动退出回到命令提示行。

```
    获取：1 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main mysql-common all
5.5.37-0ubuntu0.12.04.1 [13.3 kB]
    获取：2 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main libmysqlclient18
amd64 5.5.37-0ubuntu0.12.04.1 [945 kB]
    获取：3 http://cn.archive.ubuntu.com/ubuntu/ precise/main libnet-daemon-perl all 0.48-1
[43.1 kB]
    获取：4 http://cn.archive.ubuntu.com/ubuntu/ precise/main libplrpc-perl all 0.2020-2
[36.0 kB]
    获取：5 http://cn.archive.ubuntu.com/ubuntu/ precise/main libdbi-perl amd64 1.616-
1build2 [849 kB]
    获取：6 http://cn.archive.ubuntu.com/ubuntu/ precise/main libdbd-mysql-perl amd64
4.020- 1build2 [106 kB]
    获取：7 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main mysql-client-core-
5.5 amd64 5.5.37-0ubuntu0.12.04.1 [1,938 kB]
    获取：8 http://cn.archive.ubuntu.com/ubuntu/ precise/main libterm-readkey-perl amd64
2.30-4build3 [28.6 kB]
    获取：9 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main mysql-client-5.5
amd64 5.5.37-0ubuntu0.12.04.1 [8,338 kB]
    获取：10 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main mysql-server-core-
5.5 amd64 5.5.37-0ubuntu0.12.04.1 [6,085 kB]
    获取：11 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main mysql-server-5.5
amd64 5.5.37-0ubuntu0.12.04.1 [8,840 kB]
    获取：12 http://cn.archive.ubuntu.com/ubuntu/ precise/main libhtml-template-perl all
2.10-1 [65.0 kB]
    获取：13 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main mysql-server all
5.5.37-0ubuntu0.12.04.1 [11.5 kB]
    下载 27.3 MB，耗时 1 分 38 秒 (277 KB/s)
    正在预设定软件包 ...
    Selecting previously unselected package mysql-common.
    (正在读取数据库 ... 系统当前共安装有 261021 个文件和目录。)
    正在解压缩 mysql-common (从 .../mysql-common_5.5.37-0ubuntu0.12.04.1_all.deb) ...
    Selecting previously unselected package libmysqlclient18.
    正在解压缩 libmysqlclient18 (从 .../libmysqlclient18_5.5.37-0ubuntu0.12.04.1_
amd64.deb) ...
    Selecting previously unselected package libnet-daemon-perl.
    正在解压缩 libnet-daemon-perl (从 .../libnet-daemon-perl_0.48-1_all.deb) ...
    Selecting previously unselected package libplrpc-perl.
    正在解压缩 libplrpc-perl (从 .../libplrpc-perl_0.2020-2_all.deb) ...
    Selecting previously unselected package libdbi-perl.
    正在解压缩 libdbi-perl (从 .../libdbi-perl_1.616-1build2_amd64.deb) ...
    Selecting previously unselected package libdbd-mysql-perl.
    正在解压缩 libdbd-mysql-perl (从 .../libdbd-mysql-perl_4.020-1build2_amd64.deb) ...
    Selecting previously unselected package mysql-client-core-5.5.
    正在解压缩 mysql-client-core-5.5 (从 .../mysql-client-core-5.5_5.5.37-0ubuntu0.
12.04.1_amd64.deb) ...
    Selecting previously unselected package libterm-readkey-perl.
    正在解压缩 libterm-readkey-perl (从 .../libterm-readkey-perl_2.30-4build3_amd64.
deb) ...
    Selecting previously unselected package mysql-client-5.5.
    正在解压缩 mysql-client-5.5 (从 .../mysql-client-5.5_5.5.37-0ubuntu0.12.04.1_
amd64.deb) ...
```

```
    Selecting previously unselected package mysql-server-core-5.5.
    正在解压缩 mysql-server-core-5.5 (从 .../mysql-server-core-5.5_5.5.37-0ubuntu0.
12.04.1_amd64.deb) ...
    正在处理用于 man-db 的触发器...
    正在设置 mysql-common (5.5.37-0ubuntu0.12.04.1) ...
    Selecting previously unselected package mysql-server-5.5.
    (正在读取数据库 ... 系统当前共安装有 261398 个文件和目录。)
    正在解压缩 mysql-server-5.5 (从 .../mysql-server-5.5_5.5.37-0ubuntu0.12.04.1_
amd64.deb) ...
    Selecting previously unselected package libhtml-template-perl.
    正在解压缩 libhtml-template-perl (从 .../libhtml-template-perl_2.10-1_all.deb) ...
    Selecting previously unselected package mysql-server.
    正在解压缩 mysql-server (从 .../mysql-server_5.5.37-0ubuntu0.12.04.1_all.deb) ...
    正在处理用于 ureadahead 的触发器...
    正在处理用于 man-db 的触发器...
    正在设置 libmysqlclient18 (5.5.37-0ubuntu0.12.04.1) ...
    正在设置 libnet-daemon-perl (0.48-1) ...
    正在设置 libplrpc-perl (0.2020-2) ...
    正在设置 libdbi-perl (1.616-1build2) ...
    正在设置 libdbd-mysql-perl (4.020-1build2) ...
    正在设置 mysql-client-core-5.5 (5.5.37-0ubuntu0.12.04.1) ...
    正在设置 libterm-readkey-perl (2.30-4build3) ...
    正在设置 mysql-client-5.5 (5.5.37-0ubuntu0.12.04.1) ...
    正在设置 mysql-server-core-5.5 (5.5.37-0ubuntu0.12.04.1) ...
    正在设置 mysql-server-5.5 (5.5.37-0ubuntu0.12.04.1) ...
    140430 11:43:16 [Warning] Using unique option prefix key_buffer instead of
key_buffer_size is deprecated and will be removed in a future release.

    Please use the full name instead.
    mysql start/running, process 16533
    正在设置 libhtml-template-perl (2.10-1) ...
    正在设置 mysql-server (5.5.37-0ubuntu0.12.04.1) ...
    正在处理用于 libc-bin 的触发器...
    ldconfig deferred processing now taking place
    alloy@alloy:~/linuxshell/CH17$
```

在安装过程中会出现提示框要求设置 MySQL 的 root 用户密码，如图 17-3 所示，此时设置一个对应的密码即可，系统会让用户再次输入该密码以确定。

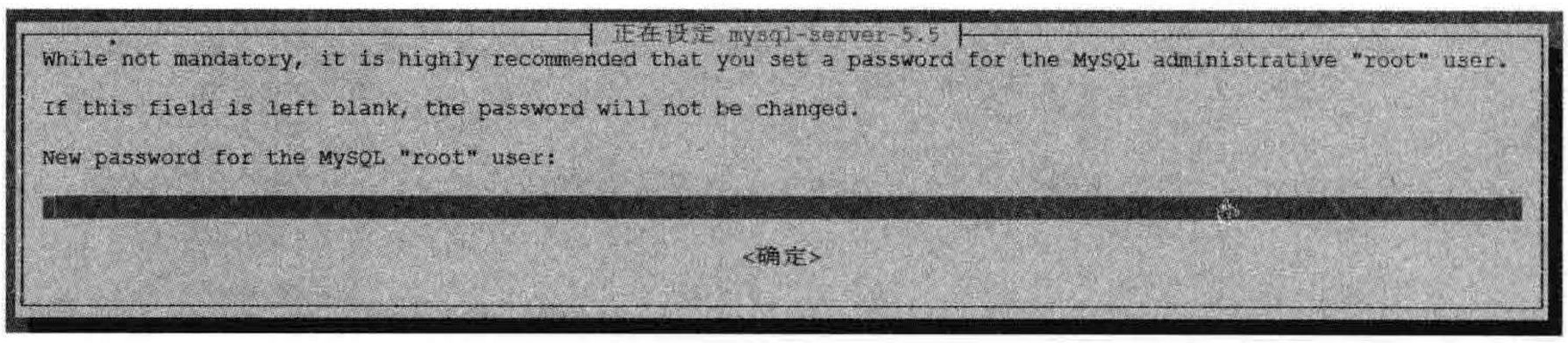


图 17-3　设置 MySQL 的密码

在MySQL安装完成之后可以使用sudo netstat -tap|grep mysql命令来查看当前系统中有没有启动对应的进程。

```
    alloy@alloy:~/linuxshell/CH17$ sudo netstat -tap|grep mysql
    tcp        0      0 localhost:mysql         *:*                     LISTEN      16533/
mysqld
```

然后使用 mysql -u root -p 命令，输入刚刚设定的密码，即可成功登录。登录 MySQL 成功表示 MySQL 安装和配置已经成功，并且 MySQL 服务器进程已经启动，这样就可以编写 Shell 脚本使用 MySQL 数据库了。

```
alloy@alloy:~/linuxshell/CH17$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 42
Server version: 5.5.37-0ubuntu0.12.04.1 (Ubuntu)

Copyright (c) 2000, 2014, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

如果需要查看 MySQL 使用的字符集，可以在登录后使用 show variables like 'character%'; 命令。

```
mysql> show variables like 'character%';
+--------------------------+----------------------------+
| Variable_name            | Value                      |
+--------------------------+----------------------------+
| character_set_client     | utf8                       |
| character_set_connection | utf8                       |
| character_set_database   | latin1                     |
| character_set_filesystem | binary                     |
| character_set_results    | utf8                       |
| character_set_server     | latin1                     |
| character_set_system     | utf8                       |
| character_sets_dir       | /usr/share/mysql/charsets/ |
+--------------------------+----------------------------+
8 rows in set (0.00 sec)
```

此外，需要注意的是，在安装完 MySQL 后，其在/etc 目录下创建了子目录 mysql，其中 my.cnf 文件为 MySQL 的配置文件，通过修改该文件的内容可以实现在 MySQL 中的配置。

```
alloy@alloy:~/linuxshell/CH17$ ls -l  /etc/mysql
总用量 16
drwxr-xr-x 2 root root 4096  4月 30 11:43 conf.d
-rw------- 1 root root  333  4月 30 11:43 debian.cnf
-rwxr-xr-x 1 root root 1220  4月 21 14:43 debian-start
-rw-r--r-- 1 root root 3507  4月 30 11:48 my.cnf
alloy@alloy:~/linuxshell/CH17$
```

17.5.2 Shell 脚本使用 MySQL

Shell 脚本若要使用 MySQL，首先需要登录数据库服务器，登录成功后才能向数据库提交 SQL 语句。下面的例 17-9 给出一个用于登录 MySQL 服务器的脚本：

```
#例 17-9: log.sh 脚本登录 MySQL 服务器
#!/bin/bash
```

```
MYSQL=`which mysql`                          #利用命令替换获得 mysql 的路径

mysql -u root -p                             #以 mysql 用户登录 MySQL 数据库
```

log.sh 脚本首先利用命令替换获得 mysql 的路径，which mysql 命令的结果一般是/usr/bin/mysql，将此结果保存到 MYSQL 变量中，然后执行该变量，-u 参数带上登录数据库的用户名，-p 选项表示需要密码验证。log.sh 脚本是其他使用 MySQL 数据库的基础，而且利用 MYSQL=`which mysql`这种方法极易扩展到其他数据库，比如：数据库服务器基于 PostgreSQL 构建时，我们仅需重新定义一个变量即可：

```
PSQL=`which psql`
$PSQL test
```

下面是 log.sh 脚本的执行结果，执行 log.sh 脚本后，提示输入密码，密码输入正确后，即可登录 MySQL 数据库。

```
#例 17-9 log.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH17$ ./log.sh
Enter password:                              #输入 MySQL 设置的 root 用户密码
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 44
Server version: 5.5.37-0ubuntu0.12.04.1 (Ubuntu)

Copyright (c) 2000, 2014, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>                                       #二级提示符出现，登录成功
```

MySQL 数据库服务器是基于数据库、表和数据来实现数据存储的，服务器上包含多个数据库，每个数据库可以包含多张表，表包含多个字段，数据是存储在表中的。在 MySQL 二级提示符（即 mysql>）下输入一些命令，可以查看当前数据库、表，下面的例 17-10 给出了查看数据库的命令：

```
#例 17-10：查看数据库和进入数据库的命令
alloy@alloy:~/linuxshell/CH17$ mysql -u root -p          #以 root 登录数据库
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 45
Server version: 5.5.37-0ubuntu0.12.04.1 (Ubuntu)

Copyright (c) 2000, 2014, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;                   #显示服务器上包含的数据库名称
+--------------------+
| Database           |
```

```
+--------------------+                          #下面是包含三个数据库的名称
| information_schema |
| mysql              |
| performance_schema |
+--------------------+
3 rows in set (0.00 sec)
mysql> create database test;                   #创建一个名称为 test 的数据库
Query OK, 1 row affected (0.00 sec)
mysql> show databases;                         #再次显示服务器上包含的数据库名称

+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| test               |                        #test 数据库已经建立
+--------------------+
4 rows in set (0.00 sec)

mysql> use test;                               #进入 test 数据库
Database changed                               #提示数据库变换成功
mysql>
```

在 MySQL 二级提示符下的命令是 MySQL 特有的命令，并不是 Shell 命令。注意，MySQL 每条命令后都需要用分号(;)结束，若没有分号，MySQL 将不执行输入的命令。show databases 命令用于显示当前服务器上所有的数据库名称，目前 MySQL 中存在三个数据库：information_schema、mysql 和 test，use test 表示选择 test 数据库，即进入 test 数据库，进入某数据库后，才能查看或新建该数据库的表。值得一提的是，MySQL 命令是不区分大小写的，例如，show databases 和 SHOW DATABASES 是等价的。

下面举一个例子，该例子在脚本中创建 MySQL 数据库表，脚本名字是 create.sh，内容如下：

```
#例 17-11: create.sh 脚本创建 people 表
#!/bin/bash

MYSQL=`which mysql`

mysql test -u root -p <<EOF                    #Here-document 用法
#创建 newpeople 表，包含 4 个属性
create table newpeople(name VARCHAR(20),sex CHAR(1),birth DATE,birthaddr VARCHAR(20));
show tables;
#可以加入任意的 MySQL 命令和 SQL 语句
EOF
```

create.sh 脚本极具参考价值，可以作为编写使用 MySQL 的 Shell 脚本的模板。mysql test 表示登录到 MySQL 服务器的 test 数据库，相当于例 17-10 中 use test 命令，-u mysql 表示以 mysql 用户的身份登录，-p 表示需要密码验证。<<EOF 是 Here-document 的用法，我们以前只介绍过 cat 命令的 Here-document 用法，数据库操作依然可以使用此用法，在两个 EOF 之间输入的任意 MySQL 命令和 SQL 语句就相当于 mysql>后面所输入的命令。因此，这种用法可以很方便地将多条 MySQL 命令和 SQL 语句提交到数据库服务器。create.sh 脚本的两个 EOF

之间有两条命令，create table newpeople 是 SQL 语句，用于创建名为 newpeople 的表，括号内定义了 newpeople 表的属性及其类型，show tables 是 MySQL 命令，用于显示当前数据库的所有表名称。限于篇幅，本书在此不介绍关于 SQL 语言、MySQL 命令的详细用法，读者可以自行参考关系数据库方面的书籍。下面给出 create.sh 脚本的执行结果：

```
#例 17-11 create.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH17$ chmod u+x create.sh
alloy@alloy:~/linuxshell/CH17$ ./create.sh
Enter password:                                    #在此输入密码
Tables_in_test
newpeople                                          #新建的 people 表
alloy@alloy:~/linuxshell/CH17$
```

create.sh 脚本执行完毕后，test 数据库中出现了 people 表，这说明 create.sh 脚本创建 people 表成功。下面再给出一个脚本，用于向 people 表中插入数据，脚本名为 insert.sh，内容如下：

```
#例 17-12: insert.sh 脚本用于向 people 表中插入数据
#!/bin/bash

MYSQL=`which mysql`

mysql test -u root -p <<EOF
#与 create.sh 脚本的区别在于 SQL 语句的不同
#向 newpeople 表插入 6 条记录，记录间用逗号隔开，最后以分号结束
insert into newpeople values ('Z Wu','m','1982-09-11','China'),
                      ('Q Cai','f','1984-11-01','China'),
                      ('T Ma','m','1981-07-17','USA'),
                      ('D Li','m','1982-09-16','China'),
                      ('J Park','m','1970-10-21','Korea'),
                      ('Z Lin','m','1983-02-20','HongKong');
select * from newpeople;                           #显示 newpeople 表中的所有数据
EOF
```

insert.sh 脚本的结构与 create.sh 脚本一样，两者的区别仅在于两个 EOF 之间的 SQL 语句不同，insert into people values 是 SQL 语句，它向 people 表插入 6 条记录，记录间用逗号隔开，最后以分号结束，select * from people 表示将 people 表中的所有数据选出，即显示 people 表中的所有数据。下面给出 insert.sh 脚本的执行结果：

```
#例 17-12 insert.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH17$ chmod u+x insert.sh
alloy@alloy:~/linuxshell/CH17$ ./insert.sh
Enter password:
name    sex     birth   birthaddr          #属性名称
Z Wu    m       1982-09-11      China      #新插入的 6 条记录
Q Cai   f       1984-11-01      China
T Ma    m       1981-07-17      USA
D Li    m       1982-09-16      China
J Park  m       1970-10-21      Korea
Z Lin   m       1983-02-20      HongKong
alloy@alloy:~/linuxshell/CH17$
```

insert.sh 脚本执行结果显示的是 select 语句的结果，这表示 insert.sh 脚本插入记录成功。因 insert.sh 脚本不够灵活，那么我们能否编写将脚本输入参数插入 people 表的脚本呢？答案是肯定的，请看下面的 insert2.sh 脚本：

```
#例 17-13: insert2.sh 脚本演示按照 name sex birth birthaddr 的顺序插入记录
```

```
#!/bin/bash

MYSQL=`which mysql`

if [ $# -ne 4 ]                                    #输入参数不能少于 4 个
then
  echo "Usage:insert2.sh name sex birth birthaddr"
else
  statement="insert into newpeople values ('$1','$2','$3','$4');"      #生成 SQL 语句
mysql test -u root -p <<EOF
$statement                                         #灵活的用法
EOF
 if [ $? -eq 0 ]                                  #判定退出码是否正常
 then
  echo "Data sucessfully added."
else
 echo "Problem adding data"
 fi
fi
```

insert2.sh 脚本能按照 name、sex、birth、birthaddr 的顺序插入记录，首先，insert2.sh 脚本判断是否有四个输入参数，若输入参数不为四个，输出 Usage:提示信息；若输入参数是四个，则生成 SQL 语句，保存于 statement 变量中。然后，将 statement 变量放到两个 EOF 之间，从而提交到 MySQL 数据库。最后，insert2.sh 脚本通过对退出码是否为零的判定，判断插入数据是否成功。下面给出例 17-13 中 insert2.sh 脚本的执行结果：

```
#例 17-13 insert2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH17$ chmod u+x insert2.sh
alloy@alloy:~/linuxshell/CH17$ ./insert2.sh Jane f 1986-07-09 Canada  #插入正确的记录
Enter password:
Data sucessfully added.
alloy@alloy:~/linuxshell/CH17$ ./insert2.sh Bob m China  #输入参数只有 3 个的情况
Usage:insert2.sh name sex birth birthaddr
alloy@alloy:~/linuxshell/CH17$
```

上面给出 insert2.sh 脚本执行正确和错误的情况，insert2.sh 脚本带四个输入参数时，能成功地将数据插入到 people 表；当 insert2.sh 脚本只有三个时，操作不成功，Shell 提示错误信息。

Shell 脚本能对数据库记录做灵活的处理，比如，要打印出 newpeople 表中所有女性的姓名和出生地，我们就可以结合 MySQL 数据库操作、管道和 awk 命令来实现，下面的 female.sh 脚本实现了这一功能：

```
#例 17-14：female.sh 脚本打印出 newpeople 表中所有女性的姓名和出生地
#!/bin/bash

MYSQL=`which mysql`

#生成 SQL 语句，用于选择出女性的记录
statement="select * from newpeople where sex='f';"

mysql test -u root -p -e "$statement"  #用-e 选项将 statement 提交到 MySQL
    | tr "[\011]" ":"                                    #将 Tab 键替换成冒号
    | awk -F":" '{printf ("%-8s\t%s\n",$1,$4)}'         #格式化打印第 1 域和第 4 域
```

female.sh 脚本用于打印出 people 表中所有女性的姓名和出生地，statement 变量保存了生

成的 SQL 语句，该语句用于选择出女性的记录；然后，将 statement 提交到 MySQL，这里没有用 Here-document 的方法，而是利用 mysql 的-e 选项。值得注意的是，statement 变量需要用引号引起来。由于 people 中人的姓名有的是两个单词，如：Q Cai，有的是一个单词，如：Jane，awk 将空格和 Tab 键都认为是分隔符，因此，直接用 awk 难以指定到姓名和出生地的域。我们用 tr 命令对记录做简单处理，即将 Tab 键替换成冒号，awk 命令以冒号为域分隔符，然后，利用 printf 语句格式化打印第 1 域和第 4 域。下面给出 female.sh 脚本的执行结果：

```
#例 17-14 female.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH17$ chmod u+x female.sh
alloy@alloy:~/linuxshell/CH17$ ./female.sh
Enter password:
name          birthaddr                          #正确输出两位女性的姓名和出生地
Q Cai         China
Jane          Canada
alloy@alloy:~/linuxshell/CH17$
```

female.sh 脚本正确输出两位女性的姓名和出生地，而且 name 域长度为 8，左对齐。female.sh 脚本对数据的处理方式将数据库操作和 Shell 文本处理相结合，这为 Shell 脚本灵活处理数据库提供了一条途径。

对数据库的操作多种多样，但 Shell 脚本的编写方式都是类似的，不同之处仅在于提交到 MySQL 数据库的 SQL 语句不同。对于不同的数据库而言，Shell 脚本的区别在于登录数据库的命令，以及提交 SQL 语句的方式。希望读者通过本节的学习能举一反三，不仅学会 MySQL 数据库的其他操作方式，而且学会编写针对其他数据库系统的 Shell 脚本。

17.6 Linux 服务器性能监控系统

Linux 服务器性能监控就是在网络环境下为管理系统及终端用户提供性能参数信息、服务器状态等，所收集到的性能参数可以为管理系统制定决策、分配和调度资源等提供依据，并利于第一时间发现服务器故障，及时恢复调整服务器系统。长期对服务器性能进行监控所获得的数据可以用来分析服务器性能的瓶颈，观察用户的行为，有助于优化服务器配置和部署。鉴于此，国际上许多大学、著名 IT 公司、高性能计算中心等部门都越来越重视服务器的性能监控，普遍在部门的服务器上部署了监控软件，并提供给用户图形化的访问界面。因此，如何监控 Linux 服务器性能、监控其哪些性能参数、如何及时更新这些性能参数及以何种形式展示给用户都是必须要考虑的重要问题。服务器性能监控的研究逐渐成为人们关注的研究领域，设计和开发完善、稳定、实用的商用性能监控系统也必将成为颇具市场潜力的软件产品。

在 Linux 系统中，可以用 uname、sysinfo、vmstat、netstat、ps、top 等 Shell 命令查看特定的系统信息，但它们都仅仅为本地用户提供系统的性能信息，不支持以 XML 等 Web 数据库格式显示这些性能信息，而且也不支持远程用户的获取和访问。

本节介绍结合使用 Ganglia、Shell 编程和 MySQL 数据库实现分布式 Linux 服务器性能监控系统，该系统能在广域范围内可扩展，能包容异构资源，并能在命名和安全方面与其他成熟的中间件集成。

17.6.1 Ganglia 简介及安装

Ganglia 是一种可扩展的分布式监控系统，主要用于监控各种网络服务器、集群系统等高性能计算机系统。Ganglia 是由加利福尼亚伯克利分校开发的开源软件，最初由美国的 NPACI 和自然基金（NSF）资助，目标是为国家普适计算基础设施——网格所建立的动态监控软件。Ganglia 监控系统由两个守护进程（Daemon）：客户端 Ganglia Monitoring Daemon (gmond)和服务端 Ganglia Meta Daemon (gmetad)，以及 Ganglia PHP Web Frontend（基于 Web 的动态访问方式）组成。gmond 模块运行于所有需要被监视的节点上，负责收集本节点的 CPU 负载、内存用量、磁盘空间等系统信息。

由于 Ganglia 的 gmond 进程可以方便地收集各节点的资源信息，国际上许多大学、著名 IT 公司、高性能计算中心所开发的监控系统大都基于 gmond 进程开发。比如，网站 http://monitor.millennium.berkeley.edu 即为加利福尼亚伯克利分校的监控系统，图 17-4 显示了它以图形化的形式展示给终端用户服务器的性能参数，它利用 Ganglia 的 gmond 进程获得 CPU 和存储方面的性能，并以图形化的 Web 界面清晰地展示了过去一小时内的 CPU 负载和内存负载情况。这种网络服务器的监控模式不仅可以方便地监控本部门的网络服务器的工作情况，而且方便了终端用户对本部门网络服务器的了解，相信会越来越流行。

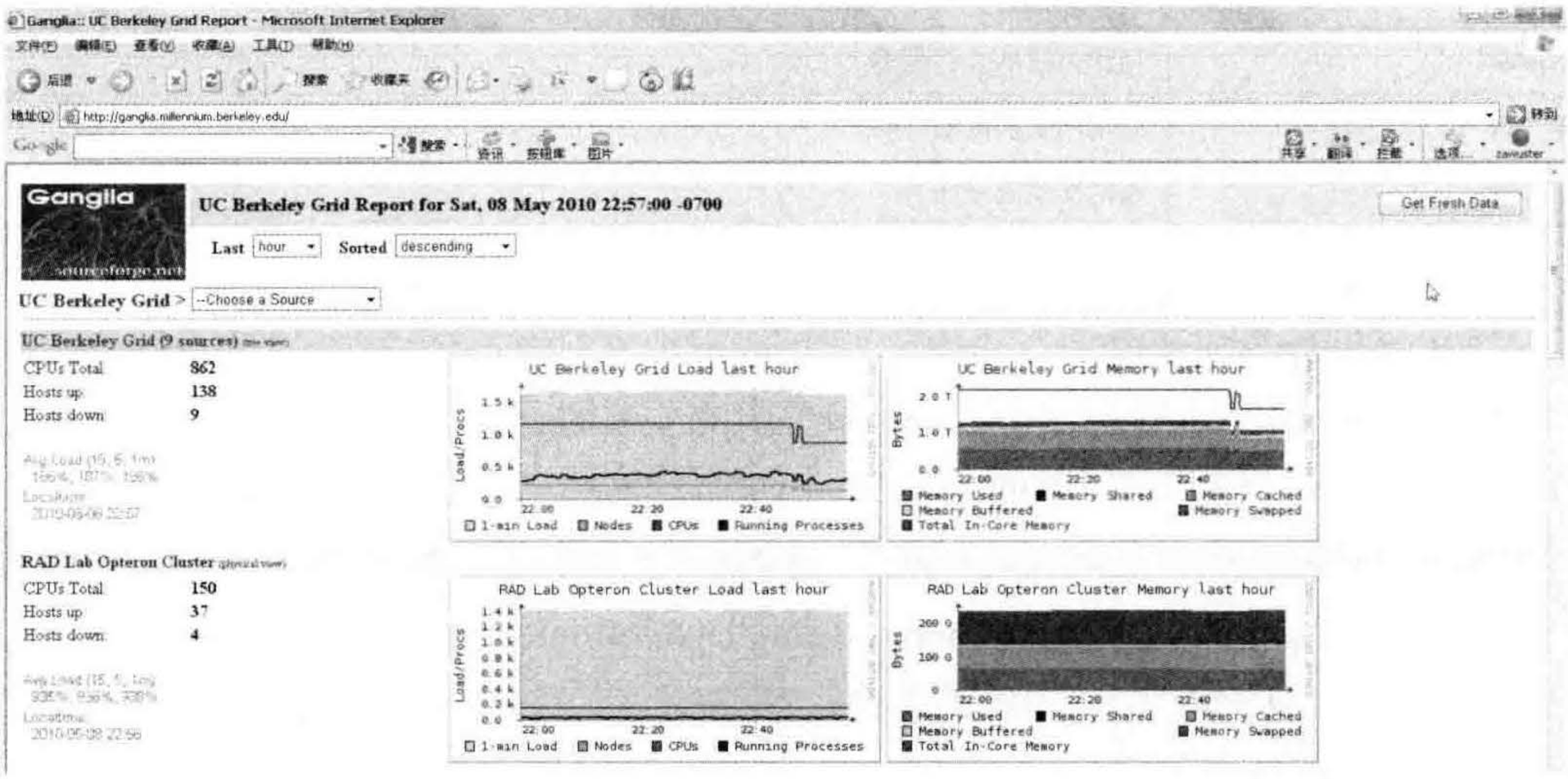


图 17-4　加利福尼亚伯克利分校的监控系统

那么，gmond 进程具体可以获取哪些性能参数呢？它所收集的性能参数大概可以分为 8 组：CPU 信息、磁盘信息、过去一段时间内的平均负载、机器类型、内存信息、网络信息、操作系统信息和交换区信息。主要参数及对应的描述如表 17-2 所示。

表 17-2　gmond 所收集的主要性能参数及描述

参数名称	意　义
cpu_idle	空闲 CPU 百分比
cpu_num	CPU 数目，实际上是 CPU 核的数目

续表

参数名称	意　义
cpu_speed	CPU 的主频，单位为 MHz
disk_free	总的磁盘空间
disk_total	空闲的磁盘空间
load_fifteen	15 分钟内的 CPU 平均负载
load_five	5 分钟内的 CPU 平均负载
load_one	1 分钟内的 CPU 平均负载
machine_type	机器类型，一般为 X_86_64
mem_buffers	缓冲区内存的大小
mem_cached	Cache 的大小
mem_free	总的物理内存大小
mem_shared	总的虚拟内存大小
mem_total	总的内存大小，物理内存加上虚拟内存
mtu	网络最大的传输包的长度，单位是 Byte
os_name	操作系统名字
os_release	操作系统的发行版本
swap_free	空闲的交换区的空间
swap_total	总的交换区空间

为了获取 Linux 服务器的性能参数，我们需要在每台待监控的 Linux 服务器上安装 Ganglia，并启动 gmond 进程。

在 Ubuntu 12.04 下同样可以使用 apt-get 工具来安装 Ganglia，首先应该安装 apache2 php5（对于安装了最新的更新包的 Ubuntu 12.04，可能已经安装完成）。

```
#例 17-15：Ganglia 的安装
alloy@alloy:~/linuxshell/CH17$ sudo apt-get install rrdtool apache2 php5
```

使用 sudo apt-get install ganglia-monitor ganglia-webfrontend gmetad 命令来安装 Ganglia。

```
alloy@alloy:~/linuxshell/CH17$ sudo apt-get install ganglia-monitor ganglia-
webfrontend gmetad
[sudo] password for alloy:
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
将会安装下列额外的软件包：
  apache2 apache2-mpm-prefork apache2-utils apache2.2-bin apache2.2-common libapache2-
mod-php5 libaprutil1-dbd-sqlite3
  libaprutil1-ldap libconfuse-common libconfuse0 libdbi1 libganglia1 librrd4 php5
php5-cli php5-common php5-gd rrdtool
  ttf-dejavu
建议安装的软件包：
  apache2-doc apache2-suexec apache2-suexec-custom php-pear php5-suhosin librrds-perl
下列【新】软件包将被安装：
  apache2 apache2-mpm-prefork apache2-utils apache2.2-bin apache2.2-common
```

```
ganglia-monitor ganglia-webfrontend gmetad
      libapache2-mod-php5  libaprutil1-dbd-sqlite3  libaprutil1-ldap  libconfuse-common
libconfuse0 libdbi1 libganglia1 librrd4 php5
      php5-cli php5-common php5-gd rrdtool ttf-dejavu
    升级了 0 个软件包，新安装了 22 个软件包，要卸载 0 个软件包，有 4 个软件包未被升级。
    需要下载 10.7 MB 的软件包。
    解压缩后会消耗掉 27.9 MB 的额外空间。
```

使用“Y”来确定安装，此时系统会自动安装下载对应的包。

```
    获取: 1 http://cn.archive.ubuntu.com/ubuntu/ precise/universe libconfuse-common all
2.7-4 [4,798 B]
    获取: 2 http://cn.archive.ubuntu.com/ubuntu/ precise/universe libconfuse0 amd64 2.7-4
[24.5 kB]
    获取: 3 http://cn.archive.ubuntu.com/ubuntu/ precise/main libaprutil1-dbd-sqlite3 amd64
1.3.12+dfsg-3 [10.4 kB]
    获取: 4 http://cn.archive.ubuntu.com/ubuntu/ precise/main libaprutil1-ldap amd64
1.3.12+dfsg-3 [8,044 B]
    获取: 5 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main apache2.2-bin amd64
2.2.22-1ubuntu1.5 [1,339 kB]
    获取: 6 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main apache2-utils amd64
2.2.22-1ubuntu1.5 [91.3 kB]
    获取: 7 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main apache2.2-common
amd64 2.2.22-1ubuntu1.5 [226 kB]
    获取: 8 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main apache2-mpm-prefork
amd64 2.2.22-1ubuntu1.5 [2,400 B]
    获取: 9 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main apache2 amd64
2.2.22-1ubuntu1.5 [1,494 B]
    获取: 10 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/universe libganglia1
amd64 3.1.7-2ubuntu1.1 [136 kB]
    获取: 11 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/universe ganglia-monitor
amd64 3.1.7-2ubuntu1.1 [54.1 kB]
    获取: 12 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main php5-common amd64
5.3.10-1ubuntu3.11 [1,797 kB]
    获取: 13 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main libapache2-mod-php5
amd64 5.3.10-1ubuntu3.11 [3,137 kB]
    获取: 14 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main php5 all
5.3.10-1ubuntu3.11 [1,076 B]
    获取: 15 http://cn.archive.ubuntu.com/ubuntu/ precise/main libdbi1 amd64 0.8.4-5.1 [28.5
kB]
    获取: 16 http://cn.archive.ubuntu.com/ubuntu/ precise/main librrd4 amd64 1.4.7-1 [242
kB]
    获取: 17 http://cn.archive.ubuntu.com/ubuntu/ precise/main rrdtool amd64 1.4.7-1 [369
kB]
    获取: 18 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/universe ganglia-
webfrontend all 3.1.7-2ubuntu1.1 [110 kB]
    获取: 19 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/universe gmetad amd64
3.1.7-2ubuntu1.1 [29.4 kB]
    获取: 20 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main php5-cli amd64
5.3.10-1ubuntu3.11 [3,051 kB]
    获取: 21 http://cn.archive.ubuntu.com/ubuntu/ precise-updates/main php5-gd amd64
5.3.10-1ubuntu3.11 [38.8 kB]
    获取: 22 http://cn.archive.ubuntu.com/ubuntu/ precise/main ttf-dejavu all 2.33-2ubuntu1
[3,178 B]
    下载 10.7 MB，耗时 13 秒 (778 KB/s)
    正在预设定软件包 ...
```

```
    Selecting previously unselected package libconfuse-common.
    (正在读取数据库 ... 系统当前共安装有 261498 个文件和目录。)
    正在解压缩 libconfuse-common (从 .../libconfuse-common_2.7-4_all.deb) ...
    Selecting previously unselected package libconfuse0.
    正在解压缩 libconfuse0 (从 .../libconfuse0_2.7-4_amd64.deb) ...
    Selecting previously unselected package libaprutil1-dbd-sqlite3.
    正在解压缩 libaprutil1-dbd-sqlite3 (从 .../libaprutil1-dbd-sqlite3_1.3.12+dfsg-3_
amd64.deb) ...
    Selecting previously unselected package libaprutil1-ldap.
    正在解压缩 libaprutil1-ldap (从 .../libaprutil1-ldap_1.3.12+dfsg-3_amd64.deb) ...
    Selecting previously unselected package apache2.2-bin.
    正在解压缩 apache2.2-bin (从 .../apache2.2-bin_2.2.22-1ubuntu1.5_amd64.deb) ...
    Selecting previously unselected package apache2-utils.
    正在解压缩 apache2-utils (从 .../apache2-utils_2.2.22-1ubuntu1.5_amd64.deb) ...
    Selecting previously unselected package apache2.2-common.
    正在解压缩 apache2.2-common (从 .../apache2.2-common_2.2.22-1ubuntu1.5_amd64.deb) ...
    Selecting previously unselected package apache2-mpm-prefork.
    正在解压缩 apache2-mpm-prefork (从 .../apache2-mpm-prefork_2.2.22-1ubuntu1.5_
amd64.deb) ...
    Selecting previously unselected package apache2.
    正在解压缩 apache2 (从 .../apache2_2.2.22-1ubuntu1.5_amd64.deb) ...
    Selecting previously unselected package libganglia1.
    正在解压缩 libganglia1 (从 .../libganglia1_3.1.7-2ubuntu1.1_amd64.deb) ...
    Selecting previously unselected package ganglia-monitor.
    正在解压缩 ganglia-monitor (从 .../ganglia-monitor_3.1.7-2ubuntu1.1_amd64.deb) ...
    Selecting previously unselected package php5-common.
    正在解压缩 php5-common (从 .../php5-common_5.3.10-1ubuntu3.11_amd64.deb) ...
    Selecting previously unselected package libapache2-mod-php5.
    正在解压缩 libapache2-mod-php5 (从 .../libapache2-mod-php5_5.3.10-1ubuntu3.11_
amd64.deb) ...
    Selecting previously unselected package php5.
    正在解压缩 php5 (从 .../php5_5.3.10-1ubuntu3.11_all.deb) ...
    Selecting previously unselected package libdbi1.
    正在解压缩 libdbi1 (从 .../libdbi1_0.8.4-5.1_amd64.deb) ...
    Selecting previously unselected package librrd4.
    正在解压缩 librrd4 (从 .../librrd4_1.4.7-1_amd64.deb) ...
    Selecting previously unselected package rrdtool.
    正在解压缩 rrdtool (从 .../rrdtool_1.4.7-1_amd64.deb) ...
    Selecting previously unselected package ganglia-webfrontend.
    正在解压缩 ganglia-webfrontend (从 .../ganglia-webfrontend_3.1.7-2ubuntu1.1_
all.deb) ...
    Selecting previously unselected package gmetad.
    正在解压缩 gmetad (从 .../gmetad_3.1.7-2ubuntu1.1_amd64.deb) ...
    Selecting previously unselected package php5-cli.
    正在解压缩 php5-cli (从 .../php5-cli_5.3.10-1ubuntu3.11_amd64.deb) ...
    Selecting previously unselected package php5-gd.
    正在解压缩 php5-gd (从 .../php5-gd_5.3.10-1ubuntu3.11_amd64.deb) ...
    Selecting previously unselected package ttf-dejavu.
    正在解压缩 ttf-dejavu (从 .../ttf-dejavu_2.33-2ubuntu1_all.deb) ...
    正在处理用于 man-db 的触发器...
    正在处理用于 ureadahead 的触发器...
    正在处理用于 ufw 的触发器...
    正在处理用于 doc-base 的触发器...
    Processing 1 added doc-base file...
    正在设置 libconfuse-common (2.7-4) ...
```

```
正在设置 libconfuse0 (2.7-4) ...
正在设置 libaprutil1-dbd-sqlite3 (1.3.12+dfsg-3) ...
正在设置 libaprutil1-ldap (1.3.12+dfsg-3) ...
正在设置 apache2.2-bin (2.2.22-1ubuntu1.5) ...
正在设置 apache2-utils (2.2.22-1ubuntu1.5) ...
正在设置 apache2.2-common (2.2.22-1ubuntu1.5) ...
Enabling site default.
Enabling module alias.
Enabling module autoindex.
Enabling module dir.
Enabling module env.
Enabling module mime.
Enabling module negotiation.
Enabling module setenvif.
Enabling module status.
Enabling module auth_basic.
Enabling module deflate.
Enabling module authz_default.
Enabling module authz_user.
Enabling module authz_groupfile.
Enabling module authn_file.
Enabling module authz_host.
Enabling module reqtimeout.
正在设置 apache2-mpm-prefork (2.2.22-1ubuntu1.5) ...
 * Starting web server apache2  apache2: Could not

    reliably determine the server's fully qualified domain name, using 127.0.1.1 for
ServerName

[ OK ]
    正在设置 apache2 (2.2.22-1ubuntu1.5) ...
    正在设置 libganglia1 (3.1.7-2ubuntu1.1) ...
    正在设置 ganglia-monitor (3.1.7-2ubuntu1.1) ...
    Adding system user: ganglia.
    Starting Ganglia Monitor Daemon: gmond.
    正在设置 php5-common (5.3.10-1ubuntu3.11) ...
    正在设置 libapache2-mod-php5 (5.3.10-1ubuntu3.11) ...

    Creating config file /etc/php5/apache2/php.ini with new version
     * Restarting web server apache2  apache2: Could not

    reliably determine the server's fully qualified domain name, using 127.0.1.1 for
ServerName
    ... waiting apache2: Could not reliably determine the server's fully qualified domain
name, using 127.0.1.1 for ServerName

[ OK ]
    正在设置 php5 (5.3.10-1ubuntu3.11) ...
    正在设置 libdbi1 (0.8.4-5.1) ...
    正在设置 librrd4 (1.4.7-1) ...
    正在设置 rrdtool (1.4.7-1) ...
    正在设置 ganglia-webfrontend (3.1.7-2ubuntu1.1) ...
    正在设置 gmetad (3.1.7-2ubuntu1.1) ...
    Starting Ganglia Monitor Meta-Daemon: gmetad.
    正在设置 php5-cli (5.3.10-1ubuntu3.11) ...
```

```
Creating config file /etc/php5/cli/php.ini with new version
update-alternatives: 使用 /usr/bin/php5 来提供 /usr/bin/php (php)于自动模式中
正在设置 php5-gd (5.3.10-1ubuntu3.11) ...
正在设置 ttf-dejavu (2.33-2ubuntu1) ...
正在处理用于 libc-bin 的触发器...
ldconfig deferred processing now taking place
```

其间，系统会提示正在设定 ganglia-webfrontend，要求重启 Web Server，选择“是”即可，如图 17-5 所示。

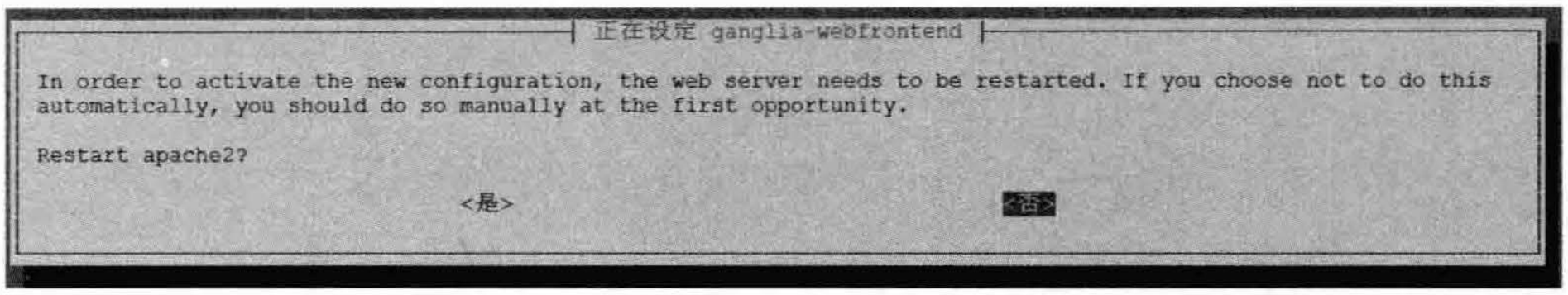


图 17-5　重启 Web Server

Ganglia 安装完毕后，就可以启动 Ganglia 的组件 gmond 了，并利用 gmond 进程获得服务器的各个性能参数。事实上，Ganglia 包括很多组件，包括客户端 Ganglia Monitoring Daemon (gmond)、服务端 Ganglia Meta Daemon (gmetad)和 Ganglia PHP Web Frontend（基于 Web 的动态访问方式），本章仅使用到 gmond 组件，有关其他组件的功能，读者可以参考 Ganglia 官方网站的相关介绍。

gmond 入口程序被默认安装在了/usr/sbin 目录下，因此，直接输入 gmond 就可以启动 gmond 进程了，由于 gmond 进程是在 TCP 的 8649 端口侦听。下面的例 17-16 给出启动 gmond 和查看其进程的命令：

```
#例 17-16：启动和查看 gmond
alloy@alloy:~/linuxshell/CH17$ gmond          #启动 gmond 进程
alloy@alloy:~/linuxshell/CH17$ sudo netstat -apl | grep 8649
#查看 gmond 进程，需要 root 权限
tcp        0      0 *:8649                  *:*                     LISTEN      19918/gmond
tcp        0      0 localhost:8649          localhost:44405         TIME_WAIT   -
tcp        0      0 localhost:8649          localhost:44407         TIME_WAIT   -
tcp        0      0 localhost:8649          localhost:44406         TIME_WAIT   -
tcp        0      0 localhost:8649          localhost:44406         TIME_WAIT   -
udp        0      0 alloy.local:34363       239.2.11.71:8649        ESTABLISHED 19918/gmond
udp        0      0 239.2.11.71:8649        *:*                                 19918/gmond
alloy@alloy:~/linuxshell/CH17$
```

从例 17-16 中 netstat 命令的结果可以看出，gmond 已经开始侦听，进程号是 18287，这表示 Ganglia 安装正常，gmond 启动正常。

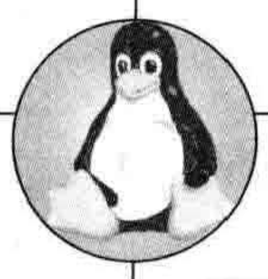
17.6.2　提取服务器性能参数名称及数据

使用 Telnet 协议可以获得 Ganglia 所收集的性能参数，一般只要使用下列命令，即可得到与本服务器在同一网段的所有服务器性能参数名称及其数据：

```
telnet localhost 8649
```

Ganglia 所产生的性能参数名称及其数据是以 XML 格式的文件存储的，因此，我们可以将 telnet localhost 8649 命令的结果重定向到文本文件，就可以利用 Shell 编程强大的文本处理功能提取出 XML 文件中存储的服务器性能参数名称及其数据。下面的例 17-17 演示了获取 Ganglia 所监控的服务器性能参数的命令：

```
#例 17-17：获取 Ganglia 所监控的服务器性能参数
#将服务器性能参数名称及其数据重定向到 gmond_msg_1.txt 文件
alloy@alloy:~/linuxshell/CH17$ telnet localhost 8649 > gmond_msg_1.txt
alloy@alloy:~/linuxshell/CH17$ cat gmond_msg_1.txt
#以下是 Ganglia 出现的系统信息，与具体性能参数无关
#因而，提取性能参数时可以忽略这段信息
Trying 127.0.0.1...
Connected to localhost.localdomain (127.0.0.1).
Escape character is '^]'.
<?xml version="1.0" encoding="ISO-8859-1" standalone="yes"?>
<!DOCTYPE GANGLIA_XML [
  <!ELEMENT GANGLIA_XML (GRID|CLUSTER|HOST)*>
    <!ATTLIST GANGLIA_XML VERSION CDATA #REQUIRED>
    <!ATTLIST GANGLIA_XML SOURCE CDATA #REQUIRED>
  <!ELEMENT GRID (CLUSTER | GRID | HOSTS | METRICS)*>
    <!ATTLIST GRID NAME CDATA #REQUIRED>
    <!ATTLIST GRID AUTHORITY CDATA #REQUIRED>
    <!ATTLIST GRID LOCALTIME CDATA #IMPLIED>
  <!ELEMENT CLUSTER (HOST | HOSTS | METRICS)*>
    <!ATTLIST CLUSTER NAME CDATA #REQUIRED>
    <!ATTLIST CLUSTER OWNER CDATA #IMPLIED>
    <!ATTLIST CLUSTER LATLONG CDATA #IMPLIED>
    <!ATTLIST CLUSTER URL CDATA #IMPLIED>
    <!ATTLIST CLUSTER LOCALTIME CDATA #REQUIRED>
  <!ELEMENT HOST (METRIC)*>
    <!ATTLIST HOST NAME CDATA #REQUIRED>
    <!ATTLIST HOST IP CDATA #REQUIRED>
    <!ATTLIST HOST LOCATION CDATA #IMPLIED>
    <!ATTLIST HOST REPORTED CDATA #REQUIRED>
    <!ATTLIST HOST TN CDATA #IMPLIED>
    <!ATTLIST HOST TMAX CDATA #IMPLIED>
    <!ATTLIST HOST DMAX CDATA #IMPLIED>
    <!ATTLIST HOST GMOND_STARTED CDATA #IMPLIED>
  <!ELEMENT METRIC EMPTY>
    <!ATTLIST METRIC NAME CDATA #REQUIRED>
    <!ATTLIST METRIC VAL CDATA #REQUIRED>
    <!ATTLIST METRIC TYPE (string | int8 | uint8 | int16 | uint16 | int32 | uint32 |
float | double | timestamp) #REQUIRED>
    <!ATTLIST METRIC UNITS CDATA #IMPLIED>
    <!ATTLIST METRIC TN CDATA #IMPLIED>
    <!ATTLIST METRIC TMAX CDATA #IMPLIED>
    <!ATTLIST METRIC DMAX CDATA #IMPLIED>
    <!ATTLIST METRIC SLOPE (zero | positive | negative | both | unspecified) #IMPLIED>
    <!ATTLIST METRIC SOURCE (gmond | gmetric) #REQUIRED>
  <!ELEMENT HOSTS EMPTY>
    <!ATTLIST HOSTS UP CDATA #REQUIRED>
    <!ATTLIST HOSTS DOWN CDATA #REQUIRED>
    <!ATTLIST HOSTS SOURCE (gmond | gmetric | gmetad) #REQUIRED>
  <!ELEMENT METRICS EMPTY>
```




```
        <!ATTLIST METRICS NAME CDATA #REQUIRED>
        <!ATTLIST METRICS SUM CDATA #REQUIRED>
        <!ATTLIST METRICS NUM CDATA #REQUIRED>
        <!ATTLIST METRICS TYPE (string | int8 | uint8 | int16 | uint16 | int32 | uint32
| float | double | timestamp) #REQUIRED>
        <!ATTLIST METRICS UNITS CDATA #IMPLIED>
        <!ATTLIST METRICS SLOPE (zero | positive | negative | both | unspecified) #IMPLIED>
        <!ATTLIST METRICS SOURCE (gmond | gmetric) #REQUIRED>
    ]>
    <GANGLIA_XML VERSION="3.0.3" SOURCE="gmond">
    <CLUSTER     NAME="unspecified"     LOCALTIME="1228044276"     OWNER="unspecified"
LATLONG="unspecified" URL="unspecified">
    #下面开始出现组播网段内所有服务器的性能参数名称及其数据
    #因而，需要从下面格式的数据中提取出有用的信息
    <HOST  NAME="seugrid2.seu.edu.cn"  IP="172.18.12.178"  REPORTED="1228044273"  TN="2"
TMAX="20" DMAX="0" LOCATION="unspecified" GMOND_STARTED="1225956735">
    <METRIC NAME="disk_total" VAL="25.618" TYPE="double" UNITS="GB" TN="3138" TMAX="1200"
DMAX="0" SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="cpu_speed" VAL="2992" TYPE="uint32" UNITS="MHz" TN="734" TMAX="1200"
DMAX="0" SLOPE="zero" SOURCE="gmond"/>
    <METRIC  NAME="part_max_used"  VAL="72.6"  TYPE="float"  UNITS=""  TN="0"  TMAX="180"
DMAX="0" SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="swap_total" VAL="1048568" TYPE="uint32" UNITS="KB" TN="734" TMAX="1200"
DMAX="0" SLOPE="zero" SOURCE="gmond"/>
    <METRIC NAME="os_name" VAL="Linux" TYPE="string" UNITS="" TN="734" TMAX="1200" DMAX="0"
SLOPE="zero" SOURCE="gmond"/>
    <METRIC NAME="cpu_user" VAL="1.4" TYPE="float" UNITS="%" TN="6" TMAX="90" DMAX="0"
SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="cpu_system" VAL="0.9" TYPE="float" UNITS="%" TN="6" TMAX="90" DMAX="0"
SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="cpu_aidle" VAL="99.0" TYPE="float" UNITS="%" TN="6" TMAX="3800" DMAX="0"
SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="load_five" VAL="0.01" TYPE="float" UNITS="" TN="67" TMAX="325" DMAX="0"
SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="proc_run" VAL="1" TYPE="uint32" UNITS="" TN="67" TMAX="950" DMAX="0"
SLOPE="both" SOURCE="gmond"/>
    <METRIC  NAME="mem_free"  VAL="265616"  TYPE="uint32"  UNITS="KB"  TN="20"  TMAX="180"
DMAX="0" SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="mem_buffers" VAL="131836" TYPE="uint32" UNITS="KB" TN="20" TMAX="180"
DMAX="0" SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="swap_free" VAL="1048440" TYPE="uint32" UNITS="KB" TN="20" TMAX="180"
DMAX="0" SLOPE="both" SOURCE="gmond"/>
    <METRIC  NAME="bytes_in"  VAL="2833.20"  TYPE="float"  UNITS="bytes/sec"  TN="260"
TMAX="300" DMAX="0" SLOPE="both" SOURCE="gmond"/>
    <METRIC  NAME="pkts_out"  VAL="10.32"  TYPE="float"  UNITS="packets/sec"  TN="260"
TMAX="300" DMAX="0" SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="cpu_num" VAL="2" TYPE="uint16" UNITS="CPUs" TN="734" TMAX="1200" DMAX="0"
SLOPE="zero" SOURCE="gmond"/>
    <METRIC  NAME="disk_free"  VAL="7.075"  TYPE="double"  UNITS="GB"  TN="0"  TMAX="180"
DMAX="0" SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="mem_total" VAL="1938436" TYPE="uint32" UNITS="KB" TN="734" TMAX="1200"
DMAX="0" SLOPE="zero" SOURCE="gmond"/>
    <METRIC NAME="cpu_wio" VAL="0.1" TYPE="float" UNITS="%" TN="6" TMAX="90" DMAX="0"
SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="boottime" VAL="1225157553" TYPE="uint32" UNITS="s" TN="734" TMAX="1200"
```

```
DMAX="0" SLOPE="zero" SOURCE="gmond"/>
    <METRIC NAME="machine_type" VAL="x86" TYPE="string" UNITS="" TN="734" TMAX="1200"
DMAX="0" SLOPE="zero" SOURCE="gmond"/>
    <METRIC NAME="os_release" VAL="2.6.18-1.2798.fc6xen" TYPE="string" UNITS="" TN="734"
TMAX="1200" DMAX="0" SLOPE="zero" SOURCE="gmond"/>
    <METRIC NAME="cpu_nice" VAL="0.0" TYPE="float" UNITS="%" TN="6" TMAX="90" DMAX="0"
SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="cpu_idle" VAL="97.7" TYPE="float" UNITS="%" TN="6" TMAX="90" DMAX="0"
SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="load_one" VAL="0.03" TYPE="float" UNITS="" TN="67" TMAX="70" DMAX="0"
SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="load_fifteen" VAL="0.00" TYPE="float" UNITS="" TN="67" TMAX="950"
DMAX="0" SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="proc_total" VAL="348" TYPE="uint32" UNITS="" TN="67" TMAX="950" DMAX="0"
SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="mem_shared" VAL="0" TYPE="uint32" UNITS="KB" TN="20" TMAX="180" DMAX="0"
SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="mem_cached" VAL="596232" TYPE="uint32" UNITS="KB" TN="20" TMAX="180"
DMAX="0" SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="gexec" VAL="OFF" TYPE="string" UNITS="" TN="118" TMAX="300" DMAX="0"
SLOPE="zero" SOURCE="gmond"/>
    <METRIC NAME="bytes_out" VAL="3473.05" TYPE="float" UNITS="bytes/sec" TN="260"
TMAX="300" DMAX="0" SLOPE="both" SOURCE="gmond"/>
    <METRIC NAME="pkts_in" VAL="13.50" TYPE="float" UNITS="packets/sec" TN="260"
TMAX="300" DMAX="0" SLOPE="both" SOURCE="gmond"/>
    </HOST>
    <HOST NAME="seugrid1.seu.edu.cn" IP="172.18.12.181" REPORTED="1228044269" TN="7"
TMAX="20" DMAX="0" LOCATION="unspecified" GMOND_STARTED="1225956458">
    ……
    #内容类似于172.18.12.178，限于篇幅，省略
    </HOST>
    <HOST NAME="seugrid5.seu.edu.cn" IP="172.18.12.177" REPORTED="1228044267" TN="8"
TMAX="20" DMAX="0" LOCATION="unspecified" GMOND_STARTED="1227854283">
    ……
    #内容类似于172.18.12.178，限于篇幅，省略
    </HOST>
    </CLUSTER>
    </GANGLIA_XML>
    alloy@alloy:~/linuxshell/CH17$
```

例 17-17 首先将服务器性能参数名称及其数据重定向到 gmond_msg_1.txt 文件，然后，查看 gmond_msg_1.txt 文件内容。gmond_msg_1.txt 文件很长，<HOST>标签之前是 Ganglia 出现的系统信息，不包含性能参数相关的有用信息；从<HOST>标签开始保存了组播网段内所有服务器的性能参数名称及其数据，每组<HOST></HOST>标签对内是一台服务器的性能参数名称及其数据，其格式都相同，只是其中的数据不一样。限于篇幅，例 17-17 中只完整地给出了一台服务器的性能参数名称及其数据，每组数据以<HOST>标签开始，<HOST>标签内首先给出该台服务器的主机名和 IP 地址，如：NAME="seugrid2.seu.edu.cn" IP="172.18.12.178"，性能参数以<METRIC>开头，包含某性能参数的名称、值、单位等信息，如：<METRIC NAME="disk_total" VAL="25.618" TYPE="double" UNITS="GB" TN="3138" TMAX="1200" DMAX="0" SLOPE="both" SOURCE="gmond"/>，该条记录的参数是总硬盘容量，为 25GB，数据类型是 double 型。

在了解了 Ganglia 输出数据之后，我们需要编写 Shell 脚本提取 gmond_msg_1.txt 文件中的有用数据，最终将这些数据存储到 MySQL 数据库，在此之前，我们需要编写一个初始化数据库的脚本，即在 MySQL 数据库中新建一张表，表的字段包含机器的名称、IP 地址，以及 gmond_msg_1.txt 文件中所有以<METRIC>开头的指标，例 17-18 给出的 initialization.sh 脚本实现了监控数据表的初始化：

```
#例 17-18：initialization.sh 脚本实现监控数据表的初始化
#!/bin/bash

MYSQL=`which mysql`

$MYSQL test -u mysql -p <<EOF
#下面是提交到 MySQL 的 SQL 语句，用于创建 resource 表
create table resource(HOSTNAME VARCHAR(30) NOT NULL default '',
                      IP  VARCHAR(20) NOT NULL default '',
                      disk_total VARCHAR(20) default NULL,
                      cpu_speed VARCHAR(20) default NULL,
                      part_max_used VARCHAR(20) default NULL,
                      swap_total VARCHAR(20) default NULL,
                      os_name VARCHAR(20) default NULL,
                      cpu_user VARCHAR(20) default NULL,
                      cpu_system VARCHAR(20) default NULL,
                      cpu_aidle VARCHAR(20) default NULL,
                      load_five VARCHAR(20) default NULL,
                      proc_run VARCHAR(20) default NULL,
                      mem_free VARCHAR(20) default NULL,
                      mem_buffers VARCHAR(20) default NULL,
                      swap_free VARCHAR(20) default NULL,
                      bytes_in VARCHAR(20) default NULL,
                      pkts_out VARCHAR(20) default NULL,
                      cpu_num VARCHAR(20) default NULL,
                      disk_free VARCHAR(20) default NULL,
                      mem_total VARCHAR(20) default NULL,
                      cpu_wio VARCHAR(20) default NULL,
                      boottime VARCHAR(20) default NULL,
                      machine_type VARCHAR(20) default NULL,
                      os_release VARCHAR(20) default NULL,
                      cpu_nice VARCHAR(20) default NULL,
                      cpu_idle VARCHAR(20) default NULL,
                      load_one VARCHAR(20) default NULL,
                      load_fifteen VARCHAR(20) default NULL,
                      proc_total VARCHAR(20) default NULL,
                      mem_shared VARCHAR(20) default NULL,
                      mem_cached VARCHAR(20) default NULL,
                      gexec VARCHAR(20) default NULL,
                      bytes_out VARCHAR(20) default NULL,
                      pks_in VARCHAR(20) default NULL);
#resource 表包含 34 个字段，HOSTNAME 和 IP 字段不能为空
show tables;
EOF
```

上述 initialization.sh 脚本与 17.5.2 节所介绍的 create.sh 脚本基本一样，只是提交到 MySQL 数据库的 SQL 语句不同，在 initialization.sh 脚本中，我们创建了 resource 表，包含 34 个字段，

IP 地址是主码，HOSTNAME 和 IP 不能为空，其他 32 个字段与 gmond_msg_1.txt 文件中以<METRIC>开头的指标相对应，所有的字段类型都为变长字符型（VARCHAR 型）。

执行 initialization.sh 脚本之后，MySQL 的 test 数据库就会出现一张新的表 resource。这样我们就可以开始编写提取 gmond_msg_1.txt 文件数据，并将其存储到 resource 表的 Shell 脚本。由于脚本比较长，我们先给出脚本内容，再详细解释，脚本名字是 monitor.sh，内容如下：

```
#例 17-19：monitor.sh 脚本提取监控数据并存储到数据库
#!/bin/bash

#预处理 gmond_msg_1.txt 文件，将该文件从第 1 行到匹配关键字<CLUSTER 的行删除
#并将删除行之后的文件保存到 machine_record 中
sed '1,/<CLUSTER/d' gmond_msg_1.txt > machine_record

record="Recording each line!"                    #用于临时保存 machine_record 的每一行内容
MYSQL=`which mysql`

#while 循环是脚本的主体部分，利用本书第 10 章所述的代码块重定向功能
#再利用 read 命令逐行读取 machine_record 的每一行
while [ -n "$record" ]                           #终止条件是 record 变量为空
do

read record
first_field=`echo $record | cut -d" " -f1`  #提取每行的第 1 域，保存到 first_field 变量

#当行的第 1 域是<HOST 时，这是一台机器的第 1 行，需要在里面提取出主机名和 IP 地址
if [ "$first_field" = "<HOST" ]
then
echo "Processing HOST and IP"
index=0                                          #数组游标，初始化为 0，表示一台机器的开始

#提取 record 的第 2 域，并只保留双引号内的值，赋给 VAR 数组的第 0 个元素
VAR[$index]=`echo $record | cut -d" " -f2 | sed -e 's/^.*=\"//' -e 's/\"//'`
let "index+=1"                                   #数组游标增加 1

#提取 record 的第 3 域，并只保留双引号内的值，赋给 VAR 数组的第 1 个元素
VAR[$index]=`echo $record | cut -d" " -f3 | sed -e 's/^.*=\"//' -e 's/\"//'`
echo ${VAR[$index]}

let "index+=1"

#当行的第 1 域是<METRIC 时，保存的是机器的具体指标
elif [ "$first_field" = "<METRIC" ]
then

#提取 record 的第 3 域，并只保留双引号内的值，赋给 VAR 数组的下一个元素
#第 3 域保存的是该指标的取值
VAR[$index]=`echo $record | cut -d" " -f3 | sed -e 's/^.*=\"//' -e 's/\"//'`
let "index+=1"

#当行的第 1 域是</HOST 时，表示一台机器的监控数据提取结束
#此时可以将 VAR 数组中的数据存储到 MySQL 的 resource 表之中
elif [ "$first_field" = "</HOST>" ]
then
```

```
echo "Writing into Database"
statement="insert into resource values ("          #statement 是提交到数据库的 SQL 语句

#for 循环自动将 VAR 数组元素插入 SQL 语句，每个元素需要用单引号引起，
#元素之间用逗号分隔
for j in ${VAR[@]}
do
 statement=$statement\'$j\',
done
#for 循环结束后，去掉 statement 最后一个逗号，加上)；两个符号
statement=${statement%,}\)\;
echo $statement
$MYSQL test -u mysql  <<EOF
$statement                          #直接将 statement 提交到数据库就能将 VAR 数组存储到 resource 表
EOF
fi

done < machine_record          #代码块重定向
```

monitor.sh 脚本比较复杂，综合使用了代码块重定向、sed、cut、管道、条件测试、命令替换、数组、变量、数据库操作等知识。首先，monitor.sh 脚本对 gmond_msg_1.txt 文件进行预处理，将该文件从第 1 行到匹配关键字<CLUSTER 的行删除，即将 Ganglia 出现的系统信息删除，便于提取下面的数据，并且定义两个变量：record 用于临时保存 machine_record 的每一行内容，当读完 machine_record 时，record 必为空，因此，while 循环的终止条件是 record 变量为空，为了能进入第 1 轮的 while 循环，record 不能为空，我们将其赋为"Recording each line!"，实际上，record 的初值可以为任意非空值；MYSQL 保存 mysql 命令路径，用于连接数据库。

while 循环是 monitor.sh 脚本的主体部分，利用本书第 10 章所述的代码块重定向功能，将 while 循环的标准输入重定向到 machine_record 文件，read 命令将每次读取 machine_record 文件的一行。然后，monitor.sh 脚本利用 if/elif/else 结构判断和处理三种情况：① 一台机器监控数据段的开始，以关键字<HOST 开头的行；② 一台机器的具体监控数据，以关键字<METRIC 开头的行；③ 一台机器监控数据段的结束，以关键字</HOST 开头的行。因此，我们测试条件的依据是 record 的第 1 域，利用命令替换和 cut 命令很容易提取出 record 的第 1 域，并存放到 first_field 变量中。

对于上述第①种情况，需要在此行提取出主机名和 IP 地址，分别是该行的第 2 域和第 3 域，利用类似于 first_field 变量的方法可以提取指定域，但是，域的内容是 NAME="seugrid2.seu.edu.cn" 和 IP="172.18.12.178"，而我们所需提取的实际数据是双引号之间的内容。因此，需要利用 sed 命令将多余部分删除，只将双引号内的值赋给 VAR 数组，VAR 数组的游标必须在此时被初始化，这样使得每换一台机器，游标 index 重新归 0。对于上述第②种情况，我们只需简单地将第 3 域赋给数组即可，方法类似于提取主机名和 IP 地址的办法。对于上述第③种情况，此时一台机器的数据全部提取出来，已经保存到 VAR 数组中。因此，我们需要将 VAR 数组内容依次写到 MySQL 的 resource 表中，数据库操作与 initialization.sh 脚本类似，最关键的操作是如何自动生成提交到 MySQL 数据库的 SQL 语句，我们分三段拼接成该 SQL 语句，保存于 statement 变量中，for 循环前面生成 statement 的前面部分"insert into

resource values (", for 循环自动将 VAR 数组元素插入 SQL 语句，每个元素需要用单引号引起，元素之间用逗号分隔，for 循环结束时，必然多了一个逗号，并需要加上右半括号“)”和分号，命令 statement=${statement%,}\)\;灵活使用 Shell 的字符串处理功能，先删除从右匹配最短的子串，再加上“);”两个符号得到。下面给出 monitor.sh 脚本的执行结果：

```
#例 17-19 monitor.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH17$ chmod u+x monitor.sh
alloy@alloy:~/linuxshell/CH17$ ./monitor.sh
Processing HOST and IP
Writing into Database
#下面是提交到数据库的 SQL 语句
insert into resource values ('seugrid2.seu.edu.cn','172.18.12.178','25.618','2992',
'72.6','1048568','Linux','1.4','0.9','99.0','0.01','1','265616','131836','1048440','283
3.20','10.32','2','7.075','1938436','0.1','1225157553','x86','2.6.18-1.2798.fc6xen','0.
0','97.7','0.03','0.00','348','0','596232','OFF','3473.05','13.50');
Processing HOST and IP
Writing into Database
insert into resource values ('seugrid1.seu.edu.cn','172.18.12.181','76.389','2793',
'33.5','1048568','Linux','2.3','0.3','98.8','0.10','0','14976','13132','594356','1553.1
3','3.22','2','50.805','513748','0.3','1225156794','x86','2.6.18-1.2798.fc6','0.0','97.
1','0.15','0.06','250','0','70636','OFF','815.85','4.85');
Processing HOST and IP
Writing into Database
insert into resource values ('seugrid5.seu.edu.cn','172.18.12.177','153.317','1200',
'15.0','4128760','Linux','0.0','0.0','99.8','0.00','0','112784','251900','4128760','396
.05','0.80','2','135.131','2072712','0.0','1225850027','x86','2.6.9-78.0.1.ELhugemem','
0.0','100.0','0.00','0.00','117','0','1220220','OFF','63.33','1.65');
alloy@alloy:~/linuxshell/CH17$
```

monitor.sh 脚本最关键的输出是提交到 MySQL 数据库的 SQL 语句，即 statement 变量值，从中可以看出脚本的运行情况，gmond_msg_1.txt 文件内共有三台机器的监控数据，monitor.sh 脚本每处理一台机器，生成一条 SQL 语句，包含 34 个具体数据，其中，主机名和 IP 地址来自第 1 行 record，其余来自以关键字<METRIC 开头的行。

17.6.3 动态更新服务器监控数据

完成 initialization.sh 脚本和 monitor.sh 脚本后，监控系统并未结束。显而易见，服务器的很多指标处于动态变化之中，如 1 分钟平均负载（load_one）、5 分钟平均负载（load_five）、剩余磁盘容量（disk_free）等指标，我们需要实现服务器监控数据的动态更新。所谓动态更新，本质上是定期利用 Ganglia 的 gmond 进程收集一次服务器监控数据，再写入数据库中，Linux 系统的定时操作就需要使用 crontab。

仅仅在 crontab 中定时执行 monitor.sh 脚本是不够的，因为 monitor.sh 脚本只负责从 Ganglia 的结果文件提取所需数据。因而，我们还需要重新写一个监控系统的入口脚本，通过该脚本封装整个监控系统，入口脚本名为 run.sh，内容如下：

```
#例 17-20：run.sh 脚本封装监控系统
#!/bin/bash

echo "Start running GMOND"
gmond                                  #启动 gmond 进程
```

```
echo "Refresh performance data of servers"
#获取 Ganglia 收集的监控数据，重定向到 gmond_msg_1.txt 文件
telnet localhost 8649 > /usr/local/shell-program/MySQL-instance/gmond_msg_1.txt
monitor.sh                              #提取出 gmond_msg_1.txt 中的数据，写入数据库
```

run.sh 启动 gmond 进程，然后利用 telnet 命令获取 Ganglia 收集的监控数据，重定向到 gmond_msg_1.txt 文件，最后执行 monitor.sh 脚本，提取出 gmond_msg_1.txt 中的数据，写入数据库。这样，我们定时执行 run.sh 脚本就可以实现服务器监控数据的动态更新了。在此，我们利用 crontab 实现每分钟执行一次 run.sh 脚本，crontab 文件的内容如下：

```
alloy@alloy:~/linuxshell/CH17$ cat /etc/crontab
SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
HOME=/
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7)  OR
#sun,mon,tue,wed,thu,fri,sat
# |  |  |  |  |
# *  *  *  *  *  command to be executed

* * * * * /usr/local/shell-program/MySQL-instance/run.sh     #每分钟执行 run.sh 脚本一次
alloy@alloy:~/linuxshell/CH17$
```

initialization.sh、monitor.sh 和 run.sh 等脚本是后台运行的脚本，它们负责将服务器监控数据收集存储到数据库，并动态更新。当然，我们编写一个图形化界面向前台用户展示服务器监控的信息，这项工作的本质是将 resource 表中的信息以 Web 页面的形式展示给用户，实现的方法多种多样，如 ASP、JSP 等，具体实现方法不属于本书的讨论范围，在此仅给出我们基于 JSP 实现的服务器性能监控的界面，如图 17-6 所示。

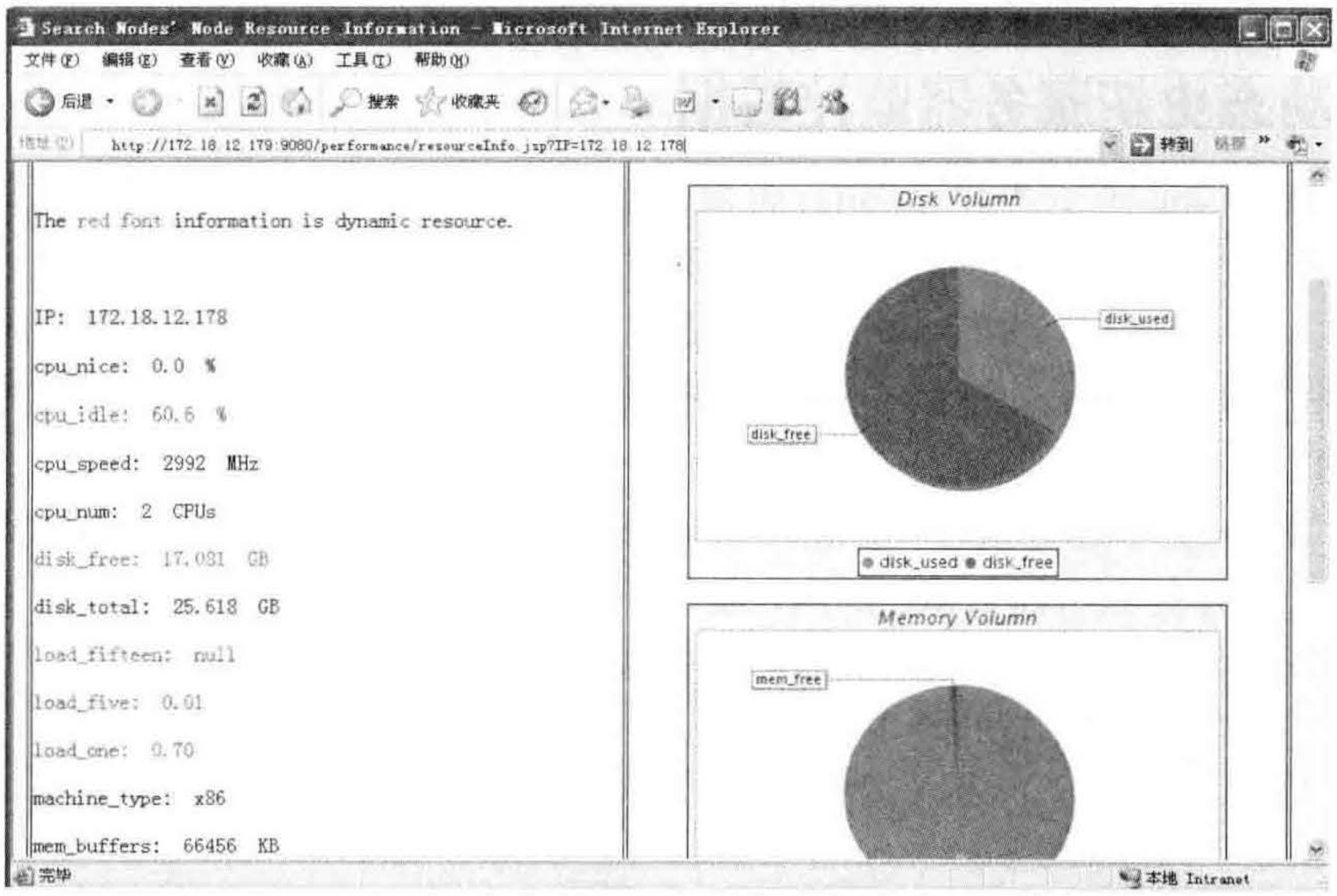


图 17-6　监控数据的前台图形化显示

图 17-6 的窗口左侧以文字形式显示了 172.18.12.178 这台服务器的性能参数；窗口右侧绘制了磁盘、内存和交换区的可用空间与剩余空间之间关系的饼图。

当然，如果读者利用更先进的插件或工具，显然可以实现更加漂亮、完善的前台界面，但是，后台脚本程序是不需要改变的，MySQL 起到了后台脚本和前台界面的桥梁作用。

17.7 本章小结

本章介绍了 Shell 编程的六个实例，涉及系统管理、文本处理和数据库等多个方面，第一个例子将 Shell 编程和 WWW 的 HTML 文本相结合，利用 sed 和 awk 两种方法实现文本文件到 HTML 文件的转化；第二个例子利用 Shell 编程解决了计算机科学中的经典问题，综合使用多种 Shell 文本处理工具，比其他高级程序设计语言更显便捷；第三个例子阐释了 Shell 编程中伪随机数的产生和应用，尤其是 Random 函数和数组索引结合的用法值得借鉴；第四个例子讲述了 Linux 的重要机制——crontab，并结合大型公司测试题说明了 crontab 在系统管理上的应用；第五个例子阐述了 Shell 脚本程序如何使用数据库，该例以 MySQL 数据库为背景，给出了几个极易扩展到其他数据库系统的脚本程序；第六个例子以当今分布式计算中的服务器监控案例为背景，基于 Ganglia，综合使用多种 Shell 编程技术，实现了一个较复杂的网络服务器性能监控系统。

17.8 上机提议

1. 参考 17.1 节给出的 htmlconver.sh 脚本，编写脚本实现将 HTML 文件转化为文本文件。

2. 17.2 节给出的 topn.sh 脚本用于查找文本中 *n* 个出现频率最高的单词，读者思考该脚本中用管道符连接的几个命令是否还可以用其他命令代替呢？（提示：第 2 个 tr 命令可用 sed 代替，head 命令也可以用 sed 代替等。）

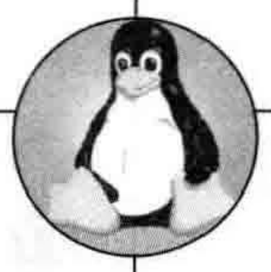
3. 编写一个字母统计程序，能将指定文件中大小写英文字母（共 52 个）出现的次数打印出来。

4. 扑克牌有 4 种花色，定义为：Clubs、Diamonds、Hearts 和 Spades，每种花色从 Ace 到 King 有 13 个取值，利用 Random 函数实现从一副扑克牌中随机取出一张牌这一操作。（提示：建议参考 random.sh 脚本，结合使用数组实现。）

5. 扩展上一题的抽取扑克牌程序，实现将一副扑克牌随机发给 4 个玩家的程序，最后输出 4 个玩家所发到牌。

6. 参考 17.3 节给出的 dice.sh 脚本，编写抛 1000 次硬币所产生的统计结果，即输出 1000 次“正面”和“反面”的次数。

7. 有一个名为 globus 的普通用户，需要在每周日凌晨零点零分定期备份/user/backup 到 /tmp 目录下，请为 globus 设计实现方案。



8．insert2.sh 脚本能将一条记录插入到 people 表中，修改 insert2.sh 脚本使其能同时插入多条记录。（提示：本题有一定难度，由于插入记录的数量不定，需要使用 shift 命令自动控制，将记录逐个移位。）

9．female.sh 脚本利用 awk 打印出姓名和出生地两个域的信息，SQL 语言的 Select 语句也可以指定打印的属性，请改变 female.sh 脚本的 Select 实现等价的功能。

10．编写一个脚本，从 17.5.2 节 test 数据库的 people 表中读取所有的数据，并将这些数据用 HTML 文件格式进行存储。

11．参考相关文档，在 Linux 上安装 PostgreSQL 数据库，并参考使用 MySQL 的脚本，编写一些脚本来使用 PostgreSQL 数据库。

12．Ganglia 产生的监控指标还包含单位、类型等数据，resource 表中却无法显示这些数据，我们可以新建一张静态表，以指标名为主码，包含单位、类型等字段，如：cpu_num-CPUs-uint16、cpu_user-%-float 等数据。请读者参考 monitor.sh 脚本，重写一个脚本建立这张表，并将 gmond_msg_1.txt 中的相关数据提取出来，存储到这张表内。

附　录

附录A POSIX 标准简介

为了实现 UNIX/Linux 操作系统的标准化，国际上的软硬件厂商、计算机组织做出了不懈的努力。POSIX 是由电气和电子工程师协会（Institute of Electrical and Electronics Engineers，IEEE）提出的一种标准，用于提高 UNIX 环境下应用程序的可移植性，从而实现 UNIX/Linux 操作系统的标准化。

POSIX 的意思是可移植操作系统接口（Portable Operating System Interface，缩写为 POSIX），是由 IEEE 和 ISO/IEC 开发的一组标准。该标准基于现有的 UNIX 实践和经验，描述了操作系统的调用服务接口，用于保证编制的应用程序可以在源代码级的多种操作系统上移植和运行。它是在 20 世纪 80 年代早期一个 UNIX 用户组的工作基础上取得的。该 UNIX 用户组原来试图将 AT&T 的 System V 操作系统和 Berkeley CSRG 的 BSD 操作系统的调用接口之间的区别重新调和集成，并于 1984 年制定了 usr/group 标准。

1985 年，IEEE 操作系统技术委员会标准小组委员会（TCOS-SS）开始在 ANSI 的支持下，责成 IEEE 标准委员会制定有关程序源代码可移植性操作系统服务接口的正式标准。到了 1986 年 4 月，IEEE 制定出了试用标准。第一个正式标准是在 1988 年 9 月批准的（IEEE 1003.1-1988），即以后经常提到的 POSIX.1 标准。

到 1989 年，POSIX 的工作被转移至 ISO/IEC 社团，并由 15 个工作组继续将其制定成 ISO 标准。到 1990 年，POSIX.1 与已经通过的 C 语言标准联合，正式批准为 IEEE 1003.1-1990（也是 ANSI 标准）和 ISO/IEC 9945-1:1990 标准。

POSIX.1 仅规定了系统服务应用程序编程接口（API），仅概括了基本的系统服务标准。因此，工作组期望对系统的其他功能也制定出标准。这样，IEEE POSIX 的工作就展开了。刚开始有 10 个批准的计划在进行，有约 300 人参加每季度为期一周的会议。着手的工作有命令与工具标准（POSIX.2）、测试方法标准（POSIX.3）、实时 API（POSIX.4）等。到了 1990 年上半年已经有 25 个计划在进行，并且有 16 个工作组参与了进来。与此同时，还有一些组织也在制定类似的标准，如 X/Open、AT&T 和 OSF 等。

在 20 世纪 90 年代初，POSIX 标准的制定正处在最后投票敲定的时候，那是 1991 年至 1993 年间，此时正是 Linux 刚刚起步的时候，这个 UNIX 标准为 Linux 提供了极为重要的信息，使得 Linux 能够在标准的指导下进行开发，并能够与绝大多数 UNIX 操作系统兼容。在最初的 Linux 内核源代码中（0.01 版、0.11 版和 0.12 版）就已经为 Linux 系统与 POSIX 标准的兼容做好了准备工作。在 Linux 0.01 版内核的 include/unistd.h 文件中就已经定义了几个有关 POSIX 标准要求的符号常数。

1991 年 7 月 3 日，Linus 在 comp.os.minix 上发布的信息就已经提到了正在搜集 POSIX 的资料。其中透露了他正在着手一个操作系统的开发，并且在开发之初已经想到要实现与 POSIX 兼容的问题。

表 A-1 描述了 POSIX 标准的主要组成部分及其功能。

表 A-1　POSIX 标准的主要组成部分及其功能描述

组成部分名称	功能描述
1003.0	管理 POSIX 开放式系统环境（OSE）。IEEE 在 1995 年通过了这项标准。ISO 的版本是 ISO/IEC 14252:1996
1003.1	被广泛接受并用于源代码级别的可移植性标准。1003.1 提供一个操作系统的 C 语言应用编程接口（API）。IEEE 和 ISO 已经在 1990 年通过了这个标准，IEEE 在 1995 年重新修订了该标准
1003.1b	一个用于实时编程的标准（以前的 P1003.4 或 POSIX.4）。这个标准在 1993 年被 IEEE 通过，被合并进 ISO/IEC 9945-1
1003.1c	一个用于线程（在一个程序中当前被执行的代码段）的标准。以前是 P1993.4 或 POSIX.4 的一部分，这个标准已经在 1995 年被 IEEE 通过，归入 ISO/IEC 9945-1:1996
1003.1g	一个关于协议独立接口的标准，该接口可以使一个应用程序通过网络与另一个应用程序通信。1996 年，IEEE 通过了这个标准
1003.2	一个应用于 Shell 和工具软件的标准，它们分别是操作系统所必须提供的命令处理器和工具程序。1992 年，IEEE 通过了这个标准，ISO 也通过了这个标准（ISO/IEC 9945-2:1993）
1003.2d	改进的 1003.2 标准
1003.5	一个相当于 1003.1 的 Ada 语言的 API。在 1992 年，IEEE 通过了这个标准，并在 1997 年对其进行了修订，ISO 也通过了该标准
1003.5b	一个相当于 1003.1b（实时扩展）的 Ada 语言的 API。IEEE 和 ISO 都已经通过了这个标准。ISO 的标准是 ISO/IEC 14519:1999
1003.5c	一个相当于 1003.1q（协议独立接口）的 Ada 语言的 API。在 1998 年，IEEE 通过了这个标准，ISO 也通过了这个标准
1003.9	一个相当于 1003.1 的 FORTRAN 语言的 API。在 1992 年，IEEE 通过了这个标准，并于 1997 年对其再次确认，ISO 也通过了这个标准
1003.10	一个应用于超级计算应用环境框架（Application Environment Profile，AEP）的标准。在 1995 年，IEEE 通过了这个标准
1003.13	一个关于应用环境框架的标准，主要针对使用 POSIX 接口的实时应用程序。在 1998 年，IEEE 通过了这个标准
1003.22	一个针对 POSIX 的关于安全性框架的指南
1003.23	一个针对用户组织的指南，主要是为了指导用户开发和使用支持操作需求的开放式系统环境（OSE）框架
2003	针对指定和使用是否符合 POSIX 标准的测试方法，有关其定义、一般需求和指导方针的一个标准。在 1997 年，IEEE 通过了这个标准
2003.1	这个标准规定了针对 1003.1 的 POSIX 测试方法的提供商要提供的一些条件。在 1992 年，IEEE 通过了这个标准
2003.2	一个定义了被用来检查与 IEEE 1003.2（Shell 和工具 API）是否符合的测试方法的标准。在 1996 年，IEEE 通过了这个标准。除了 1003 和 2003 家族以外，还有几个其他的 IEEE 标准，例如，1224 和 1228，它们也提供开发可移植应用程序的 API

附录B 常用 ASCII 码对照表

信息在计算机上是用二进制数表示的，这种表示法让人很难理解。因此，计算机上都配有输入和输出设备，这些设备的主要目的就是以一种人们可阅读的形式将信息在这些设备上

显示出来供人阅读理解。为保证人和设备、设备和计算机之间能进行正确的信息交换，人们编制了统一的信息交换代码，这就是 ASCII 码表。ASCII 码（American Standard Code for Information Interchange，美国标准信息交换码）是由美国国家标准局（ANSI）制定的，它已被国际标准化组织（ISO）定为国际标准，称为 ISO646 标准。

ASCII 码定义从 0 到 127 共 128 个数字所代表的英文字母或一样的结果和意义。由于只使用 7 个位元（bit）就可以表示从 0 到 127 的数字，大部分的电脑都使用 8 个位元来存取字元集（character set），所以，从 128 到 255 之间的数字可以用来代表另一组 128 个符号，称为 extended ASCII。其中第 0～32 号及第 127 号（共 34 个）是控制字符或通信专用字符，如控制符：LF（换行）、CR（回车）、FF（换页）、DEL（删除）、BEL（振铃）等；通信专用字符：SOH（文头）、EOT（文尾）、ACK（确认）等。第 33～126 号（共 94 个）是字符，其中，第 48～57 号为 0～9 共 10 个阿拉伯数字；65～90 号为 26 个大写英文字母；97～122 号为 26 个小写英文字母；其余为一些标点符号、运算符号等。为了便于查询，表 B-1 列出了常用的 ASCII 码对照表及其含义。

表 B-1　ASCII 码值对应表

十进制数	八进制数	十六进制数	缩写/字符	键盘对应键	解　　释
0	00	00	NUL	Control-@	空字符
1	01	01	SOH	Control-A	标题开始
1	02	02	STX	Control-B	正文开始
3	03	03	ETX	Control-C	正文结束
4	04	04	EOT	Control-D	传输结束
5	05	05	ENQ	Control-E	请求
6	06	06	ACK	Control-F	收到通知
7	07	07	BEL	Control-G	响铃
8	10	08	BS	Control-H	退格
9	11	09	HT	Control-I	水平制表符
10	12	0A	LF	Control-J	换行键
11	13	0B	VT	Control-K	垂直制表符
12	14	0C	FF	Control-L	换页键
13	15	0D	CR	Control-K	回车键
14	16	0E	SO	Control-L	不用切换
15	17	0F	SI	Control-M	启用切换
16	20	10	DLE	Control-N	数据链路转义
17	21	11	DC1	Control-O	设备控制 1
18	22	12	DC2	Control-P	设备控制 2
19	23	13	DC3	Control-Q	设备控制 3
20	24	14	DC4	Control-R	设备控制 4

续表

十进制数	八进制数	十六进制数	缩写/字符	键盘对应键	解　　释
21	25	15	NAK	Control-S	拒绝接收
22	26	16	SYN	Control-T	同步空闲
23	27	17	ETB	Control-U	传输块结束
24	30	18	CAN	Control-V	取消
25	31	19	EM	Control-W	介质中断
26	32	1A	SUB	Control-X	替补
27	33	1B	ESC	Control-Y	溢出
28	34	1C	FS	Control-Z	文件分隔符
29	35	1D	GS	-	分组符
30	36	1E	RS	-	记录分离符
31	37	1F	US	-	单元分隔符
32	40	20	Space	Spacebar	空格
33	41	21	!	!	
34	42	22	“	“	
35	43	23	#	#	
36	44	24	$	$	
37	45	25	%	%	
38	46	26	&	&	
39	47	27	‘	‘	
40	50	28	(	(	
41	51	29	)	)	
42	52	2A	*	*	
43	53	2B	+	+	
44	54	2C	‘	‘	
45	55	2D	-	-	
46	56	2E	.	.	
47	57	2F	/	/	
48	60	30	0	0	
49	61	31	1	1	
50	62	32	2	2	
51	63	33	3	3	
52	64	34	4	4	
53	65	35	5	5	
54	66	36	6	6	
55	67	37	7	7	
56	70	38	8	8	

续表

十进制数	八进制数	十六进制数	缩写/字符	键盘对应键	解　　释
57	71	39	9	9	
58	72	3A	:	:	
59	73	3B	;	;	
60	74	3C	<	<	
61	75	3D	=	=	
62	76	3E	>	>	
63	77	3F	?	?	
64	100	40	@	@	
65	101	41	A	A	
66	102	42	B	B	
67	103	43	C	C	
68	104	44	D	D	
69	105	45	E	E	
70	106	46	F	F	
71	107	47	G	G	
72	110	48	H	H	
73	111	49	I	I	
74	112	4A	J	J	
75	113	4B	K	K	
76	114	4C	L	L	
77	115	4D	M	M	
78	116	4E	N	N	
79	117	4F	O	O	
80	120	50	P	P	
81	121	51	Q	Q	
82	122	52	R	R	
83	123	53	S	S	
84	124	54	T	T	
85	125	55	U	U	
86	126	56	V	V	
87	127	57	W	W	
88	130	58	X	X	
89	131	59	Y	Y	
90	132	5A	Z	Z	
91	133	5B	[	[	
92	134	5C	\	\	
93	135	5D	]	]	

续表

十进制数	八进制数	十六进制数	缩写/字符	键盘对应键	解　　释
94	136	5E	^	^	
95	137	5F	_	_	
96	140	60	`	`	
97	141	61	a	a	
98	142	62	b	b	
99	143	63	c	c	
100	144	64	d	d	
101	145	65	e	e	
102	146	66	f	f	
103	147	67	g	g	
104	150	68	h	h	
105	151	69	i	i	
106	152	6A	j	j	
107	153	6B	k	k	
108	154	6C	l	l	
109	155	6D	m	m	
110	156	6E	n	n	
111	157	6F	o	o	
112	160	70	p	p	
113	161	71	q	q	
114	162	72	r	r	
115	163	73	s	s	
116	164	74	t	t	
117	165	75	u	u	
118	166	76	v	v	
119	167	77	w	w	
120	170	78	x	x	
121	171	79	y	y	
122	172	7A	z	z	
123	173	7B	{	{	
124	174	7C	|	|	
125	175	7D	}	}	
126	176	7E	~	~	
127	177	7F	DEL	Delete	删除键

附录C Linux 信号及其意义

由本书第 12 章的内容可知，信号是在软件层面上对中断机制的一种模拟，Linux 系统一共有 64 种信号，kill -l 命令列出的信号中，编号为 1～31 的信号是传统 UNIX 支持的信号，是不可靠信号（非实时的）；编号为 32～63 的信号是后来扩充的，称为可靠信号（实时信号）。不可靠信号和可靠信号的区别在于前者不支持排队，可能会造成信号丢失，而后者不会丢失信号。

表 C-1 列出了 Linux 信号、值、处理动作以及发出信号的原因，处理动作用字母标识，一共有以下 6 种动作：

- A 默认的动作是终止进程。
- B 默认的动作是忽略此信号。
- C 默认的动作是终止进程并进行内核映像转存（dump core）。
- D 默认的动作是停止进程。
- E 信号不能被捕获。
- F 信号不能被忽略。

表 C-1　Linux 信号、值、处理动作以及发出信号的原因

信号名称	值	处理动作	发出信号的原因
SIGHUP	1	A	终端挂起，或控制进程终止
SIGINT	2	A	键盘中断，如：break 键被按下
SIGQUIT	3	C	键盘退出键被按下
SIGILL	4	C	非法指令
SIGABRT	6	C	由 abort(3)发出的退出指令
SIGFPE	8	C	浮点异常
SIGKILL	9	AEF	kill 信号
SIGSEGV	11	C	无效的内存引用
SIGPIPE	13	A	管道破裂，写一个没有读端口的管道
SIGALRM	14	A	由 alarm(2)发出的信号
SIGTERM	15	A	终止信号，网管 bitscn_com
SIGUSR1	10	A	用户自定义信号 1
SIGUSR2	12	A	用户自定义信号 2
SIGCHLD	17	B	子进程结束信号
SIGCONT	18	-	进程继续（曾被停止的进程）
SIGSTOP	19	DEF	终止进程
SIGTSTP	20	D	控制终端（tty）上按下停止键
SIGTTIN	21	D	后台进程企图从控制终端读

续表

信号名称	值	处理动作	发出信号的原因
SIGTTOU	22	D	后台进程企图从控制终端写
SIGBUS	7	C	总线错误（错误的内存访问）
SIGPROF	27	A	Profiling 定时器到
SIGSYS	-	C	无效的系统调用（SVID）
SIGTRAP	5	C	跟踪/断点捕获
SIGURG	23	B	Socket 出现紧急条件
SIGVTALRM	26	A	实际时间报警时钟信号
SIGXCPU	24	C	超出设定的 CPU 时间限制
SIGXFSZ	25	C	超出设定的文件大小限制
SIGIOT	6	C	I/O 捕获指令，与 SIGABRT 等价
SIGSTKFLT	16	A	协处理器堆栈错误
SIGIO	29	A	某 I/O 操作现在可以执行了
SIGCLD	-	A	与 SIGCHLD 等价
SIGPWR	30	A	电源故障
SIGINFO	-	A	与 SIGPWR 等价，电源故障
SIGLOST	-	A	文件锁丢失
SIGWINCH	28	B	窗口大小改变
SIGUNUSED	31	A	未使用的信号

表 C-1 中，“处理动作”列的“-”表示信号尚未实现，所给出的值是在 i386 架构下的信号值，而在 Alpha、Sparc 等架构下，值和信号的对应关系会稍有变化。处理动作 A 终止程序是指进程退出；处理动作 B 忽略此信号指将该信号丢弃，不做任何处理；处理动作 C 内核映像转存是指将进程数据在内存的映像和进程在内核结构中存储的部分内容以一定格式转存到文件系统，并且进程退出执行，这将使得它们可以得到进程当时执行的数据值；处理动作 D 是指进入停止状态后还能重新进行下去，一般是在调试过程中。

附录D bash 内建变量索引

bash 内建变量是 Shell 编程的常用内容，本书介绍了很多内建变量，表 D-1 归纳了 bash 的内建变量，并给出本书涉及该变量的章序号，便于读者查询。

表 D-1　bash 内建变量索引表

命　令	所 属 章	值
BASH	9	记录了 bash Shell 的路径，通常为/bin/bash

续表

命　令	所 属 章	值
BASH_SUBSHELL	9，12	子 Shell 的层次
BASH_VERSINFO	9	是一个数组，包含 6 个元素，这 6 个元素用于表示 bash 的版本信息
BASH_ENV	-	用于非交互式 Shell 的初始化文件的路径名
BASH_VERSION	9	记录了 Linux 系统的 bash Shell 版本
CDPATH		cd 命令的搜索路径
COLUMNS		select 命令使用的显示宽度
DIRSTACK	9	目录栈的栈顶值
FCEDIT		fc 默认使用的编辑器名称
GLOBIGNORE	9	通配（globbing）时忽略的文件名集合
GROUPS	9	记录了当前用户所属的群组
HISTFILE		保存历史列表文件的路径名（默认为~/.bash_history）
HISTFILESIZE		保存在 HISTFILE 中的最大项数
HISTSIZE		保存在历史列表中的最大项数
HOME	6	用户主目录的路径名，用作 cd 命令的默认参数或用在字符（~）表达式中
IFS	6	内部字段分隔符，用于分词
INPUTRC	-	Readline 初始化文件的路径名（默认为~/.inputrc）
LANG	-	没有用 LC_*变量特别设置时的区域目录
LC_*	-	指定了区域目录包括 LC_COLLATE、LC_CTYPE、LC_MESSAGES 和 LC_NUMBERIC 的一组变量，使用内置命令 locale 可以显示值的列表
LINES	-	select 使用的显示高度
MAIL	11	保存用户邮件的文件的路径名
MAILCHECK	11	以秒为单位定义了 bash 检查邮件的频度
MAILPATH	11	bash 检查邮件文件的路径名列表，名字之间用冒号隔开
OLDPWD	6	记录旧的工作目录
OSTYPE	9	记录了操作系统类型
PATH	6	bash 查找命令的目录路径名列表，名字之间用冒号隔开
PPID	6	创建当前进程的进程号
PROMPT_COMMAND	-	bash 在显示主提示符之前要执行的命令
PS1	9	提示符 1，主提示符（默认为“\s-\v\$”）
PS2	9	提示符 2，辅助提示符（默认为“>”）
PS3	9	select 发出的提示符
PS4	16	bash 调试符
PWD	6	记录当前的目录路径
REPLY	9	保存 read 接受的行，还用于 select
SECONDS	9	记录脚本从开始执行到结束所耗费的时间，以秒为单位

续表

命　令	所 属 章	值
SHELL	6	保存默认的 Shell
SHELLOPTS	9	记录了处于“开”状态的 Shell 选项（options）列表
SHLVL	9	记录了 bash Shell 嵌套的层次
TMOUT	9	用于设置 Shell 的过期时间
USER	6	已登录用户的名字
UID	6	已登录用户的 ID

附录E bash 内建命令索引

本书介绍的内建命令分散到了全书的各章节，表 E-1 归纳了 bash 的内建命令，并给出本书涉及该命令的章序号，便于读者查询。

表 E-1　bash 内建命令索引表

命　令	所 属 章	含　义
!	7	保留字，逻辑非
:	12	不做任何事，只作为参数展开
.	9	读取文件并在当前 Shell 中执行它
alias	14	设置命令或命令行的别名
bg	12	将作业置于后台运行
bind	-	将关键字序列与 readline 函数或宏绑定
break	8	保留字，跳出 for、while、until、select 循环
builtin	-	调用命令的内置命令格式，而禁用同名的函数，或者同名的扩展命令
case	7	保留字，多重选择
cd	2	切换当前工作目录
command	-	找出内置和外部命令；寻找内置命令而非同名函数
continue	7	保留字，到达下一次 for、while、until、select 循环
declare	9	声明变量，定义变量属性
dirs	-	显示当前存储目录列表
disown	12	将作业从表中移除
do	8	保留字，for、while、until、select 循环的一部分
done	8	保留字，for、while、until、select 循环的一部分
echo	7	打印参数
elif	7	保留字，if 结构的一部分
else	7	保留字，if 结构的一部分

续表

命　令	所 属 章	含　义
enable	-	开启和关闭内置命令
esac	7	保留字，case 的一部分
eval	10	将参数作为命令再次处理一遍
exec	10	以特定程序取代 Shell 或为 Shell 改变 I/O
exit	7	退出 Shell
export	6	将变量声明为环境变量
fc	12	与命令历史一起运行
fg	12	将作业置于前台运行
fi	7	保留字，if 结构的一部分
for	8	保留字，for 循环的一部分
function	13	定义一个函数
getops	15	处理命令行选项
help	2	显示内置命令的帮助信息
history		显示历史命令信息
if	7	保留字，if 结构的一部分
in	8	保留字，case 的一部分
jobs	12	显示在后台运行的作业
kill	12	向进程传送信号
let	7	使变量执行算术运算
local	13	定义局部变量
logout	-	从 Shell 中注销
popd	9	从目录栈中弹出目录
pushd	9	将目录压入目录栈
pwd	6	显示当前工作目录
read	7	从标准输入中读入一行
readonly	6	将变量定义为只读
return	13	从函数或脚本中返回
select	8	保留字，生成选择菜单
set	9	设置 Shell 选项
shift	15	变换命令行参数
suspend	12	中止 Shell 的执行
test	7	评估条件表达式
then	7	保留字，if 结构的一部分

续表

命　令	所 属 章	含　义
time	9	保留字，输出统计出来的命令执行时间，其输出格式由 TIMEFORMAT 变量来控制
trap	12，16	设置信号捕捉程序
type	-	确认命令的源
typeset	9	声明变量，定义变量属性，与 declare 等价
ulimit	-	设置和显示进程占用资源的限制
umask	2	设置和显示文件权限码
unalias	14	取消别名定义
unset	6，14	取消变量或函数的定义
until	8	保留字，一种循环结构
wait	12	等待后台作业完成
while	8	保留字，一种循环结构

参 考 文 献

[1] John Bambenek, Agnieszka Klus. grep Pocket Reference. O'Reilly Media, Inc, 2009

[2] Arnold Robbins. sed & awk Pocket Reference. O'Reilly & Associates, 2000

[3] Amold Robbins. Effective awk Programming, 3rd Edition. O'Reilly & Associates, 2001

[4] Cameron Newham. Learning the bash Shell, 3rd Edition. O'Reilly & Associates, 2000

[5] AnderwS Tanenbaum. 现代操作系统. 英文版第 3 版. 北京：机械工业出版社，2009

[6] Mendel Cooper. Advanced Bash-Scripting Guide. http://www.tldp.org/LDP/abs/html

[7] David Tansley. Linux 与 UNIX Shell 编程指南. 徐焱，张春萌等译. 北京：机械工业出版社，2000

[8] Arnold Robbins, Nelson H. F. Beebe. Shell 脚本学习指南. O'Rilly Taiwan 公司编译. 北京：机械工业出版社，2009

[9] Setphen G. Kochan, Patrick Wood. UNIX Shell 编程. 袁科萍等译. 北京：中国铁道出版社，2004

[10] Richard Blum. Linux Command Line and Shell Scripting. Wiley Publishing, Inc, 2008

[11] Anatole Olczak. Korn Shell: Unix and Linux Programming Manual, 3rd Edition. Addison Wesley, 2000

[12] Red Hat Software, Inc. Linux Complete Command Reference. Sams, 1997

封面
书名
版权
前言
目录
正文